Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Гантөмөрийн Алтай

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Бакалаврын төгсөлтийн ажил

Улаанбаатар хот

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Компьютерын ухааны салбар

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Мэргэжлийн индекс: 071405000000002306
Мэргэжил: Программ хангамжийн инженерчлэл

Удирдагч: Магистр (M.Sc) Э.Батцэцэг
Зөвлөгч: Доктор (Ph.D), Дэд профессор Г.Ганбат
Гүйцэтгэгч: Г.Алтай

Улаанбаатар хот
2026 он 6 сар

Батлав. Компьютерын ухааны салбарын эрхлэгч:

..... /Доктор (Ph.D), дэд профессор А.Хүдэр/

Удирдагч:

..... /Магистр (M.Sc) Э.Батцэцэг/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Сэдэв: "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт"

№	Ажлын бүлэг, хэсгийн нэр	Эзлэх хувь	Дуусах хугацаа
1	Удиртгал хэсэг	5%	2026-2-4
2	Судалгааны хэсэг	15%	2026-2-21
3	Шинжилгээний хэсэг	15%	2026-3-4
4	Зохиомжийн хэсэг	20%	2026-4-1
5	Хөгжүүлэлтийн хэсэг	40%	2026-4-29
6	Дүгнэлт	5%	2026-5-27

Төлөвлөгөөг боловсруулсан оюутан: ..... /Г.Алтай/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Удирдагчийн гарын үсэг
1	Удиртгал	2026-2-4	
2	Судалгааны хэсэг	2026-2-21	
3	Шинжилгээний хэсэг	2026-3-4	
4	Зохиомжийн хэсэг	2026-4-1	
5	Хөгжүүлэлтийн хэсэг	2026-4-29	
6	Дүгнэлт	2026-5-27	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: ..... /Магистр (M.Sc) Э.Батцэцэг/

ЗӨВШӨӨРӨЛ

Оюутан Г.Алтай-н бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: ..... /Доктор (Ph.D), дэд профессор А.Хүдэр/

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

ШҮҮМЖИЙН ХУУДАС

Компьютерын ухааны салбарын төгсөх курсийн оюутан Г.Алтайгын "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт" сэдэвт төгсөлтийн ажлын шүүмж:

1. Төслөөр дэвшүүлсэн асуудал, үүнтэй холбоотой онолын материал уншиж судалсан байдал; Энэ талаар хүмүүсийн хийсэн судалгаа, түүний үр дүнг уншиж тусгасан эсэх:

   .....
   .....
   .....
   .....
   .....
   .....
   .....

2. Төслийн ерөнхий агуулга, шийдсэн зүйлс, хүрсэн үр дүн; Өөрийн санааг гарган, харьцуулалт хийн, дүгнэж байгаа чадвар:

   .....
   .....
   .....
   .....
   .....
   .....
   .....

3. Эмх цэгцтэй, стандарт хангасан өөрөөр хэлбэл диплом бичих шаардлагуудыг биелүүлсэн эсэх; Төсөлд анзаарагдсан алдаанууд, зөв бичгийн болон өгүүлбэр зүйн гэх мэт /Хуудас дугаарлагдаагүй, зураг хүснэгтийн дугаар, тайлбар байхгүй, шрифт хольсон, хувилсан зүйл ихээр оруулсан/:

.....

.....

.....

.....

.....

.....

4. Төслөөр орхигдуулсан болон дутуу болсон зүйлс; Цаашид анхаарах хэрэгтэй зүйлс:

.....

.....

.....

.....

.....

.....

5. Төслийн талаар онцолж тэмдэглэх зүйлс:

.....

.....

.....

.....

.....

.....

6. Ерөнхий оноо (5 оноо)

.....

Шүүмж бичсэн: ..... /Магистр (M.Sc) О.Нэр/

Ажлын газар: .....

Хаяг (Утас) .....

Зохиогч эрхийн хамгаалал

Миний бие Г.Алтай, "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт" сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ гол эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

"*Thanks to my solid academic training, today I can write hundreds of words on virtually any topic without possessing a shred of information, which is how I got a good job in journalism.*"

Dave Barry

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Хураангуй

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Г.Алтай
bbyyydriver@gmail.com

Түлхүүр үгс: Үүлэн хадгалалтын систем, веб систем, IOS native мобайл app

Талархал

Энэхүү дипломын ажлыг хийж дуусгахад хамгийн гол нөлөө үзүүлсэн, сэдэв бүрийг нягталж, үнэтэй зөвөлгөө өгч, алдааг маань засаж, удирдан чиглүүлж байсан удирдагч багш Э.Батцэцэг багшдаа хамгийн түрүүнд чин сэтгэлээсээ баярлалаа гэж хэлмээр байна. Мөн дипломын ажлын зөвлөх багш болох Г.Ганбат багшдаа үнэт цагаа зарцуулж, зааж, сургаж, орхигдуулсан зүйлсийг сануулж, хэрэгтэй зөвлөгөөнүүд өгч байсанд гүн талархлаа илэрхийлье.

Намайг бакалаврын дипломын ажил бичиж чадах хэмжээнд хүртэл зааж сургасан МХТС -н Компьютерын Ухааны салбарын багш нар, “Программ хангамж” мэргэжлийн хөтөлбөрийн хүрээнд хичээл зааж байсан ШУТИС -н багш нартаа баярлалаа. Суралцахад аюулгүй, цэвэрхэн орчныг бүрдүүлж өгч байсан МХТС -н ажилчдад баярлалаа.

Академик мэдлэгээ практик дээр туршин, дипломын ажлын хүрээнд шинэ технологиудыг судлан, мэргэжлийн хувьд туршлагажих боломж олгож байсан Грэйпсити Монгол компанийн “Технелогийн Газар” -н захирал болон ахлах инженертээ талархлаа илэрхийлмээр байна.

Эцэст нь, би дипломын ажилдаа их цаг зарцуулах боломжийг олгож, олон талаар чухал дэмжлэг үзүүлж байсан гэр бүлийнхэн болон урам зориг, эрч хүчээр урамшуулж байсан найзууддаа ч бас чин сэтгэлийн талархлаа илэрхийлэх ёстой.

Товчилсон үгс

Гарчиг

Зохиогч эрхийн хамгаалал **i**

Хураангуй **iii**

Талархал **iv**

Товчилсон үгс **v**

Удиртгал **1**

1 Хувийн үүлэн хадгалалтын системийн тухай онол, арга зүйн судалгаа **4**

1.1 Үүлэн хадгалалтын системийн ерөнхий ойлголт 4
1.2 Ижил төст системүүдийн судалгаа 6
1.2.1 Судалгааны үнэлгээний шалгуур 6
1.2.2 Нийтийн үүлэн хадгалалтын системүүд 6
Google Drive: архитектур ба шийдлүүд 7
Apple iCloud: архитектур ба шийдлүүд 7
Microsoft OneDrive: архитектур ба шийдлүүд 8
1.2.3 Хувийн үүлэн хадгалалтын системүүд 11
Nextcloud 12
Nextcloud-ийг Docker орчинд ажиллуулсан туршилтын хэсэг . . 13
1.2.4 Peer-to-Peer өгөгдөл синхрончлолын системүүд 14
1.2.5 Харьцуулсан шинжилгээ 15
1.3 Ашиглах технологиуд 16
1.4 Бүлгийн дүгнэлт 17

2 Хувийн үүлэн хадгалалтын системийн шинжилгээ **18**

2.1 Системийн үйл ажиллагааны тухай дэлгэрэнгүй 18
2.2 Функциональ шаардлага 18
2.2.1 Usecase диаграмм 19
2.2.2 Юзкейсийн тодорхойлолт 21
Хэрэглэгч үүсгэх (UC-001) 21
Хэрэглэгч устгах (UC-002) 22
Эрх олгох (UC-003) 23
Системд нэвтрэх (UC-004) 24
Файл болон хавтас харах (UC-005) 25
Файл байршуулах (UC-006) 26

Файл татаж авах (UC-007) 27
Хавтас үүсгэх (UC-008) 28
Файл/хавтас устгах (UC-009) 29
Файл/хавтас шилжүүлэх (UC-010) 30
2.3 Функциональ бус шаардлага 30
2.4 Ажлын хүрээ 31
2.4.1 Scope (Хамрах хүрээ) 31
2.4.2 Out of Scope (Хийхгүй зүйлс) 31
2.5 Бүлгийн дүгнэлт 32

3 Хувийн үүлэн хадгалалтын системийн зохиомж ба хөгжүүлэлт 33
3.1 Системийн архитектур 33
3.2 Зохиомжийн үндсэн компонентууд 33
3.3 Өгөгдлийн ерөнхий схем 34
3.4 Системийн прототип 35
3.5 Хөгжүүлсэн системийн интерфейс 38
3.6 Системд хийгдсэн тест 38
3.7 Системийн нэвтрүүлэлт 38
3.8 Бүлгийн дүгнэлт 38

Ерөнхий дүгнэлт 40

A Серверийн API-ийн жишээ 41
A.1 REST endpoint-ууд 41

Ашигласан материалын жагсаалт 42

Зургийн жагсаалт

1.1 Хэрэглэгчдийн ашиглаж буй үүлэн хадгалалтын үйлчилгээний үзүүлэлт 5
1.2 Google Colossus control plane архитектурын зураг (эх сурвалж: Google Cloud Blog) [2] 7
1.3 Apple CloudKit zone ба key management загвар (эх сурвалж: Apple Support) [9] 8
1.4 Microsoft OneDrive sync process architecture (эх сурвалж: Microsoft) [13] 10
1.5 Өргөн ашиглагдаж буй үүлэн хадгалалтын үйлчилгээнүүд 11
1.6 Хувийн сервер дээр ашиглах боломжтой үүлэн хадгалалтын платформууд 12
1.7 Nextcloud системийн өгөгдлийн давхаргын нэгдсэн шийдэл 13
1.8 Nextcloud-ийг Docker ашиглан асаасны дараа нийт RAM-ийн хэрэглээ . 14
1.9 Nextcloud-ийу веб хэрэглэгчийн интерфэйс 14

2.1 Usecase диаграм 20

3.1 Системийн архитектур 33
3.2 Хувийн үүлэн хадгалалтын системийн ERD 35
3.3 Анхны тохиргооны дэлгэц 36
3.4 Нэвтрэх дэлгэц 36
3.5 Энгийн хэрэглэгчийн үндсэн хуудас 37
3.6 Админ хэрэглэгчийн нүүр хуудас 37

Хүснэгтийн жагсаалт

1.1 Нийтийн үүлэн платформуудын архитектур ба шийдлүүдийн харьцуулалт . . . 11
1.2 Ижил төст системүүдийн харьцуулсан шинжилгээ . . . 15

2.1 UC-001 usecase-ийн тодорхойлолт . . . 21
2.2 UC-002 usecase-ийн тодорхойлолт . . . 22
2.3 UC-003 usecase-ийн тодорхойлолт . . . 23
2.4 UC-004 usecase-ийн тодорхойлолт . . . 24
2.5 UC-005 usecase-ийн тодорхойлолт . . . 25
2.6 UC-006 usecase-ийн тодорхойлолт . . . 26
2.7 UC-007 usecase-ийн тодорхойлолт . . . 27
2.8 UC-008 usecase-ийн тодорхойлолт . . . 28
2.9 UC-009 usecase-ийн тодорхойлолт . . . 29
2.10 UC-010 usecase-ийн тодорхойлолт . . . 30

Удиртгал

Өнөөгийн дижитал эрин үед үүлэн хадгалалт (Cloud Storage) нь хэрэглэгчдийн өдөр тутмын амьдралын салшгүй хэсэг болжээ. Хэрэглэгчид зураг, бичлэг, ажлын болон хувийн чухал файлуудаа интернетийн тусламжтайгаар хаанаас ч хандаж ашиглах шаардлагатай байдаг. Энэхүү хэрэгцээг хангахын тулд Google Drive, iCloud, OneDrive зэрэг төвлөрсөн үүлэн хадгалалтын үйлчилгээ өргөн хэрэглэгддэг болсон.

Гэвч эдгээр үйлчилгээний хувьд хэрэглэгчийн өгөгдөл тухайн үйлчилгээ үзүүлэгч компанийн сервер дээр хадгалагддаг бөгөөд хэрэглэгч өөрийн мэдээллийн бүрэн эзэмшил, хяналтыг алдах эрсдэлтэй байдаг. Мөн хадгалах багтаамж нэмэгдэх тусам тогтмол төлбөр төлөх шаардлага гардаг нь урт хугацаанд хэрэглэгчдэд эдийн засгийн дарамт учруулж болзошгүй.

Иймд хэрэглэгч өөрийн эзэмшлийн техник хангамжийг ашиглан өгөгдлөө хадгалах боломжтой, өөрөөр хэлбэл өөрийн сервер дээр суурилсан хувийн үүлэн хадгалалтын системийг хөгжүүлэх нь чухал асуудал болж байна. Энэхүү ажлын хүрээнд хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах, веб болон мобайл клиентээр дамжуулан удирдах боломжтой, хөнгөн бүтэцтэй (lightweight) хувийн үүлэн хадгалалтын системийг хөгжүүлэхийг зорьж байна.

Зорилго

Энэхүү төслийн зорилго нь хэрэглэгч өөрийн эзэмшлийн техник хангамжид суурилсан, өөрөө байршуулж (self-hosted) ажиллуулах боломжтой хувийн үүлэн хадгалалтын системийг зохион бүтээж хэрэгжүүлэхэд оршино. Тус систем нь хэрэглэгчдэд веб болон мобайл клиент ашиглан өөрийн сервер дээр хадгалагдсан файлуудыг удирдах, хандах боломжийг олгоно.

Төслийн зорилтууд

Дэвшүүлсэн зорилгод хүрэхийн тулд дараах зорилтуудыг хэрэгжүүлнэ. Үүнд:

1. Хувийн үүлэн хадгалалтын системүүдийн архитектур болон технологийн харьцуулсан судалгаа хийх;
2. Нөөц багатай төхөөрөмж (Raspberry Pi зэрэг) дээр ажиллах боломжтой хөнгөн серверийн программ хангамжийг зохион бүтээх;
3. Файл хадгалах, татах, зохион байгуулах үндсэн үйлдлүүдийг дэмжих серверийн REST API хөгжүүлэх;

4. Хэрэглэгч веб хөтөч ашиглан файлуудаа удирдах боломжтой веб клиент аппликейшн хөгжүүлэх;
5. iOS үйлдлийн системд зориулсан хэрэглэгчийн мобайл аппликейшн боловсруулах;
6. Хэрэглэгч өөрийн серверийг интернэт орчинд байршуулан алсаас хандах боломжийг бүрдүүлэх архитектурын шийдлийг боловсруулах.

Судлах үндэслэл

Сүүлийн жилүүдэд үүлэн хадгалалтын хэрэглээ огцом өсөж, 2025 оны байдлаар дэлхий даяар үүлэн хадгалалтын хэрэглэгчдийн тоо 2.3 тэрбумыг [1] давсан байна. Гэвч төвлөрсөн үүлэн үйлчилгээний хэрэглээ өсөхийн хэрээр дараах асуудлууд тулгарч байна.

1. **Өгөгдлийн эзэмшил ба нууцлал**: Хэрэглэгчийн мэдээлэл гуравдагч этгээдийн сервер дээр хадгалагддаг тул өгөгдлийн бүрэн хяналт хэрэглэгчид байхгүй.
2. **Тогтмол үйлчилгээний зардал**: Үүлэн хадгалалтын үйлчилгээний багтаамж нэмэгдэх тусам хэрэглэгч тогтмол төлбөр төлөх шаардлагатай болдог.

Иймд хэрэглэгч өөрийн техник хангамжийг ашиглан өгөгдлөө хадгалах боломжтой, өөрөөр хэлбэл self-hosted хувийн үүлэн системийг хөгжүүлэх нь технологийн болон практик ач холбогдолтой юм.

Судлагдсан байдал

Хувийн үүлэн хадгалалтын системийн чиглэлээр Nextcloud, Seafile, Syncthing зэрэг нээлттэй эхийн төслүүд өргөн хэрэглэгддэг. Эдгээр системүүд нь хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах боломжийг олгодог боловч ихэвчлэн том хэмжээний системийн архитектуртай, олон төрлийн нэмэлт функцүүдийг агуулсан байдаг.

Тухайлбал, Nextcloud нь хамтын ажиллагаа, календарь, мэйл зэрэг олон төрлийн үйлчилгээг нэг дор багтаасан цогц платформ бол Seafile нь файл синхрончлолын өндөр гүйцэтгэлд төвлөрсөн систем юм. Гэвч эдгээр системүүдийг бага нөөцтэй төхөөрөмж дээр ажиллуулахад тохиргоо хийхэд төвөгтэй, системийн шаардлага өндөр байх зэрэг сул талууд ажиглагддаг.

Иймд энэхүү төслөөр илүү хөнгөн бүтэцтэй, хувийн хэрэглээнд зориулсан үүлэн хадгалалтын системийг зохион бүтээхийг зорьсон.

Шинэлэг тал

Хувийн үүлэн хадгалалтын системийн шинэлэг тал:

1. **Хөнгөн архитектур**: Нөөц багатай төхөөрөмжүүд дээр ажиллах боломжтой энгийн бөгөөд үр ашигтай серверийн архитектур ашиглана.
2. **Self-hosted шийдэл**: Хэрэглэгч өөрийн сервер дээр системийг байршуулж өгөгдлөө бүрэн хянах боломжтой.
3. **Олон төрлийн клиент**: Веб болон мобайл клиент ашиглан файлуудад хандах боломжтой.

Технологийн ач холбогдол

Энэхүү систем нь технологийн хувьд дараах ач холбогдолтой:

1. **Өгөгдлийн бүрэн эзэмшил**: Хэрэглэгч өөрийн сервер дээр өгөгдлөө хадгалснаар мэдээллийн бүрэн хяналтыг хадгална.
2. **Хөнгөн серверийн шийдэл**: Орчин үеийн асинхрон серверийн архитектур ашиглан өндөр гүйцэтгэлтэй систем бий болгоно.
3. **Олон платформын клиент**: Веб болон мобайл клиентүүдээр дамжуулан хэрэглэгчийн хүртээмжийг нэмэгдүүлнэ.

Хамрах хүрээ

Тус төсөл нь дараах хүрээг хамарна:

- Өөрийн өгөгдлийн нууцлалыг эрхэмлэдэг хувь хүн болон гэр бүлийн хэрэглэгчид;
- Өндөр өртөг бүхий үүлэн үйлчилгээнээс татгалзаж, өөрийн нөөцийг ашиглах хүсэлтэй технологи сонирхогчид;

БҮЛЭГ 1

Хувийн үүлэн хадгалалтын системийн тухай онол, арга зүйн судалгаа

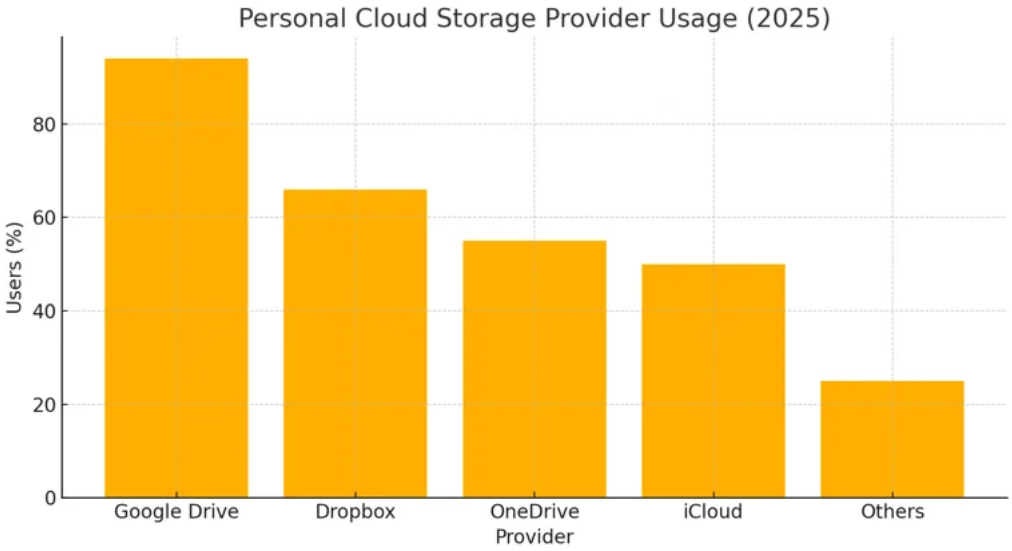
1.1 Үүлэн хадгалалтын системийн ерөнхий ойлголт

Үүлэн хадгалалт (*Cloud Storage*) нь өгөгдлийг локал төхөөрөмж дээр бус, сүлжээгээр холбогдсон алсын сервер дээр хадгалах технологи юм. Хэрэглэгч интернэтээр дамжуулан өөрийн файлуудад хаанаас ч, ямар ч төхөөрөмжөөс хандах боломжтой байдаг. Энэхүү технологи нь өгөгдлийг төвлөрсөн байдлаар хадгалах, нөөцлөх, хуваалцах зэрэг давуу талуудыг олгодог.

Үүлэн хадгалалтын системийг ерөнхийд нь дараах гурван төрөлд ангилдаг:

- **Нийтийн (Public Cloud)** – Гуравдагч байгууллагын эзэмшлийн сервер дээр өгөгдлийг хадгалдаг (жишээ нь: Google Drive, Dropbox). Хэрэглэхэд хялбар боловч өгөгдлийн хяналт харьцангуй бага байдаг.
- **Хувийн (Private Cloud)** – Хэрэглэгч өөрийн сервер дээр системийг ажиллуулж, өгөгдлөө бүрэн хянах боломжтой. Аюулгүй байдал болон нууцлал өндөр боловч тохиргоо, удирдлага шаарддаг.
- **Холимог (Hybrid Cloud)** – Нийтийн болон хувийн үүлэн системийг хослуулсан хэлбэр. Уян хатан байдал сайтай боловч архитектурын хувьд илүү төвөгтэй.

Сүүлийн жилүүдэд өгөгдлийн нууцлал, аюулгүй байдлын асуудал нэмэгдэж байгаатай холбоотойгоор хувийн үүлэн хадгалалтын системд анхаарах хандлага нэмэгдэж байна. Хэрэглэгч өөрийн өгөгдлийг бүрэн хянах, гуравдагч этгээдээс хамааралгүй байх шаардлага нь ийм төрлийн системийг хөгжүүлэх үндэслэл болж байна. 2025 оны судалгаагаар [1] дэлхий дээр нийт 2.3 тэрбум хүн үүлэн хадгалалтын үйлчилгээг гуравдагч системээс авч байгаа гэсэн судалгаа гарсан. Үүний хэрэглэгчдийн ашиглалтын үзүүлэлт болон заах зээлийн хуваалт нь Google Drive 94%, Dropbox 66%, OneDrive 55%, iCloud 50% (хэрэглэгчид эдгээр платформуудыг зэрэг хэрэглэсэн байна). Дараах зурагт хэрэглээний түвшинг графикаар харуулав.



Зураг 1.1: Хэрэглэгчдийн ашиглаж буй үүлэн хадгалалтын үйлчилгээний үзүүлэлт

Судалгааны үр дүнгээс үзэхэд хэрэглэгчдийн 55% дор хаяж 3 үүлэн хадгалалтын үйлчилгээг зэрэг хэрэглэж байна. Хүмүүс ихэвчлэн файлуудаа платформуудад хуваан, тухайн үйлчилгээний онцлогт тааруулан ашигладаг. Google Photos-ийг зураг хадгалахдаа ашиглаж, баримт бичгийг Dropbox, iPhone утасныхаа өгөгдлийг iCloud дээр хадгалах нь түгээмэл ажиглагдаж байна. Хэрэглэгчдийн дунд өгөгдлийн аюулгүй байдлын халдлага болон асуудал энэхүү салбарт хамгийн том асуудлуудын нэгт орж байгаа бөгөөд үүнд зориулж үйлчилгээ үзүүлэгч байгууллагууд илүү найдвартай шифрлэлт, хоёр шатлалт баталгаажуулалт гэх мэт хамгаалалтын шийдлүүдийг санал болгож байна. 2024 оны Англи улсад хэрэглэгчдээс авсан судалгаагаар үүлэн хадгалалтын хэрэглэгчдийн 43% нь тухайн үүлэн хадгалалтын үйлчилгээ үзүүлэгч байгууллага нь өөрсдийнх нь хувийн өгөгдлийг шинжилдэг гэдэгт санаа зовнидог гэсэн хариулт өгсөн байна. Үүлэн хадгалалтын үйлчилгээнүүд нь ойрын ирээдүйд хиймэл оюунтай ашиглан файл системийн автомат цэгцлэлт, ухаалаг хайлт, өгөгдөл шошгожуулах, олон платформ дээр дэмжигдэх байдал, сүлжээгүй үед хэрхэн sync хийгдэх гэсэн зүйлс дээр нэмэлт сайжруулалт хийгдэн өрсөлдөөн үүсэх төлөвтэй байгаа ажээ. **Энэхүү дипломын ажлаар миний бие хувийн гэсэн ангилалд багтах үүлэн хадгалалтын системийг хөгжүүлэхийг зорьж байгаа юм.**

1.2 Ижил төст системүүдийн судалгаа

Хувийн өгөгдөл хадгалах, синхрончлох системүүд нь сүүлийн жилүүдэд хэрэглэгчийн өгөгдөлд бүрэн хяналттай байх, аюулгүй байдлыг нэмэгдүүлэх, мөн нийтийн cloud-тай төстэй хэрэглээний энгийн туршлагыг хадгалах чиглэлд эрчимтэй хөгжиж байна. Энэ хүрээнд нийтийн үүлэн үйлчилгээ, self-hosted үүлэн платформ, peer-to-peer синхрончлолын хандлагуудыг харьцуулан судлав.

1.2.1 Судалгааны үнэлгээний шалгуур

Ижил төст системүүдийг дараах үндсэн шалгууруар үнэлэв.

1. **Өгөгдлийн өмчлөл ба хяналт**: өгөгдөл хэний дэд бүтэц дээр хадгалагдаж, хэн бодит удирдлагыг хэрэгжүүлж байгааг үнэлэх.
2. **Суулгалт ба арчилгааны төвөгшил**: анхны тохиргоо, шинэчлэлт, нөөцлөлт, ажиллагааны хяналтын хүндрэл.
3. **Аюулгүй байдлын загвар**: дамжуулалт болон нэвтрэлтийн хамгаалалт, эрхийн удирдлагын нарийвчлал.
4. **Сүлжээний ажиллах зарчим**: төв серверээс хамаарах эсэх, peer discovery/relay ашиглах нөхцөл.
5. **Хэрэглээний туршлага**: өдөр тутмын хэрэглээнд ойлгомжтой байдал, автоматжуулалтын түвшин.

1.2.2 Нийтийн үүлэн хадгалалтын системүүд

Нийтийн үүлэн хадгалалтын системүүдэд дараах өргөн хэрэглэгддэг платформууд орно:

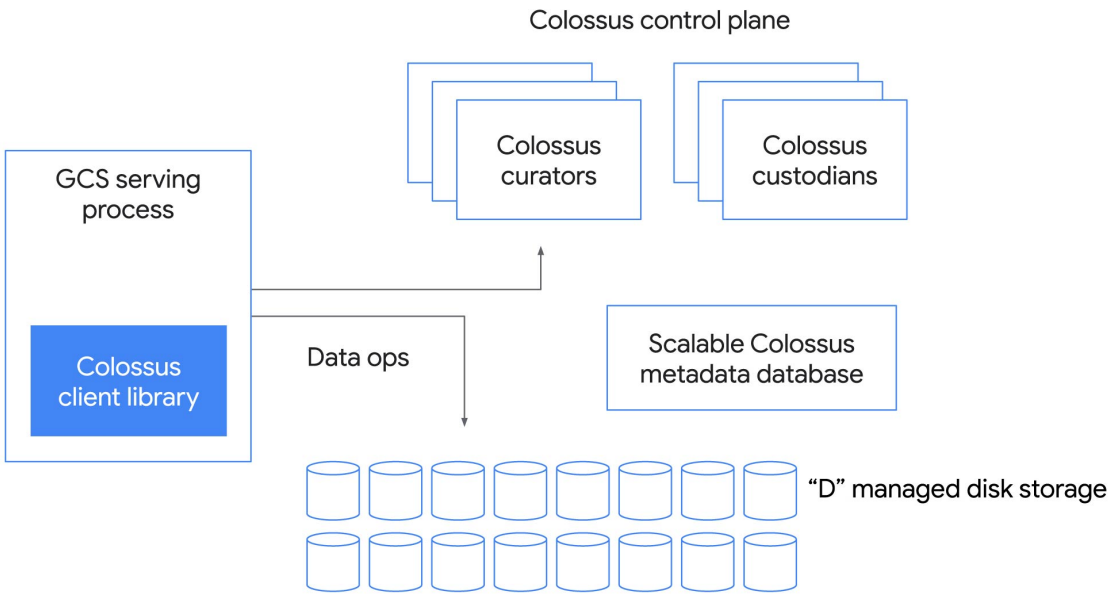
1. Apple iCloud
2. Google Drive
3. Microsoft OneDrive

Нийтийн үүлэн хадгалалтын платформуудын дотоод архитектур нь үйлдвэрлэгч тус бүр дээр хаалттай байдаг ч нийтэд нээлттэй техникийн баримт бичгүүдээс дараах үндсэн давхаргуудыг ялган дүгнэж болно: (1) клиент ба синхрончлолын давхарга, (2) мета өгөгдөл болон эрхийн удирдлагын control plane, (3) тархмал хадгалалтын data plane, (4) аюулгүй байдал ба сэргээн босголтын давхарга.

Google Drive: архитектур ба шийдлүүд

Google-ийн нийтэлсэн дэд бүтцийн тайлбараас үзэхэд Drive үйлчилгээ нь Colossus, Spanner, Borg зэрэг суурь бүрэлдэхүүнүүд дээр түшиглэсэн олон давхаргат архитектуртай. Colossus нь файл хадгалалтын үндсэн data plane-г хангаж, Spanner нь эрх, байршил зэрэг мета өгөгдлийн удирдлагыг, Borg нь нөөцийн зохион байгуулалтыг гүйцэтгэдэг [2]. Мөн Workspace-ийн аюулгүй байдлын баримт бичигт дурдсанаар өгөгдөл нь дамжуулалтын үед болон хадгалалтын үед шифрлэгдэж, олон дата төвд зэрэгцээ (live/synchronous) хуулбарлагдан, тасалдал гарсан үед автоматаар өөр дата төв рүү шилжих байдлаар өндөр хүртээмжийг хангадаг [3].

Drive-ийн API түвшинд синхрончлол ба өөрчлөлтийн менежмент нь ‘changes‘ feed, push notification (‘watch‘), мөн resumable upload механизмаар хэрэгждэг нь тасалдсан сүлжээ, том файл, олон төхөөрөмжийн зэрэг өөрчлөлт зэрэг практик нөхцөлд алдаанд тэсвэртэй ажиллах боломж бүрдүүлдэг [4, 5, 6].



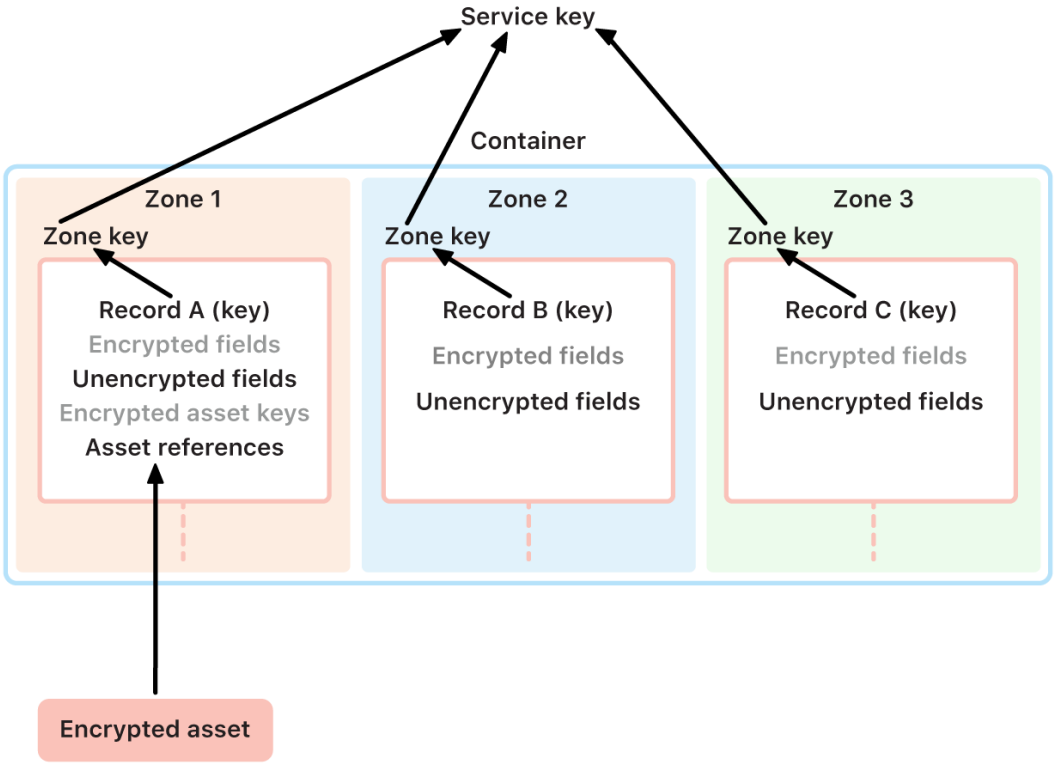
Зураг 1.2: Google Colossus control plane архитектурын зураг (эх сурвалж: Google Cloud Blog) [2]

Apple iCloud: архитектур ба шийдлүүд

iCloud нь үйлчилгээний түвшинд Apple Account-аар нэвтрэлт, төхөөрөмж хоорондын автомат синхрончлол, iCloud Drive/Backup зэрэг өгөгдлийн үйлчилгээ, мөн түлхүүрийн удирдлагын давхаргаас бүрдэнэ. Apple Platform Security баримт бичигт iCloud өгөгдлийн ихэнх нь төхөөрөмж дээр эхэлж шифрлэгдэн cloud руу дамждаг,

дамжуулалтын үед TLS, хадгалалтын үед нэмэлт шифрлэлт хэрэглэдэг гэж тайлбарласан [7]. Мөн Advanced Data Protection-ийг идэвхжүүлснээр end-to-end encryption ашиглах өгөгдлийн ангилал 14-өөс 23 болж өсдөг нь хэрэглэгчийн талын түлхүүрийн хяналтыг нэмэгдүүлдэг [7].

Хөгжүүлэгчийн түвшинд CloudKit нь контейнер, өгөгдлийн сан, zone гэсэн логик тусгаарлалтаар апп тус бүрийн өгөгдлийг салган удирдах бөгөөд хувийн өгөгдөлд шифрлэлт, төхөөрөмж хоорондын синк, схемийн менежмент зэрэг боломжуудыг санал болгодог [8]. Энэ нь iCloud-ийн архитектурт multi-tenant орчинд өгөгдөл тусгаарлах болон масштаблах чухал шийдэл болж өгдөг.



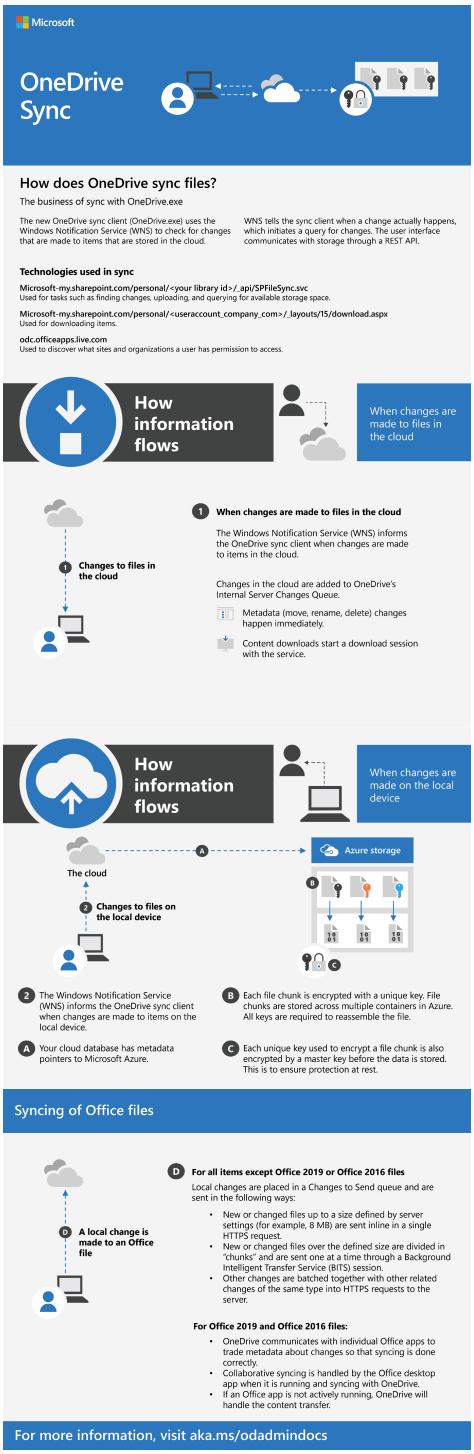
Зураг 1.3: Apple CloudKit zone ба key management загвар (эх сурвалж: Apple Support) [9]

Microsoft OneDrive: архитектур ба шийдлүүд

OneDrive-ийн аюулгүй хадгалалтын архитектур нь OneDrive болон SharePoint-д нийтлэг ашиглагддаг шифрлэлт ба хадгалалтын олон давхар хамгаалалт дээр суурилдаг. Microsoft-ийн техникийн тайлбарт өгөгдөл дамжих үед TLS, хадгалалтын үед BitLocker болон per-file encryption зэрэгцэн хэрэглэгдэж, файл болон өөрчлөлтийн хэсгүүдийг blob store-д тархаан хадгалдаг гэж тодорхойлсон [10]. Файлыг буцаан

угсрах зураглал (map) болон түлхүүрүүдийг тусдаа агуулахуудад хадгалдаг бөгөөд Blob store, Content Database, Key Store нь физикийн хувьд тусгаарлагдсан бүтэцтэй тул нэг цэг алдагдсан ч бүх өгөгдлийг шууд сэргээх боломжгүй байдлаар хамгаалдаг [10].

Үйлчилгээний түвшинд OneDrive нь Files On-Demand, синк клиент, version history, recycle bin зэрэг шийдлүүдээр хэрэглээний энгийн байдлыг хадгалдаг [11]. Мөн Microsoft Support-ийн баримтад өгөгдөл дор хаяж хоёр Azure region-д толин хуулбараар байрших, тасалдал болон ransomware-ийн үед буцаалт хийх хамгаалалтууд ашиглагддаг гэж дурдсан нь тасралтгүй ажиллагаа ба сэргээн босголтын чадамжийг нэмэгдүүлдэг [12].



Зураг 1.4: Microsoft OneDrive sync process architecture (эх сурвалж: Microsoft) [13]

Платформ	**Архитектурын цөм**	**Найдвартай байдлын шийдэл**	**Аюулгүй байдлын шийдэл**
Google Drive	Colossus, Spanner, Borg дээр суурилсан олон давхаргат бүтэц	Олон дата төвийн давхардсан хуулбарлалт, failover, автоматаар сэргэлт	Дамжуулалтын болон хадгалалтын шифрлэлт, Workspace-ийн хандалтын хяналт
Apple iCloud	iCloud services ба CloudKit (container/DB/zone) загвар	Төхөөрөмж хоорондын автомат синк, сэргээх бодлого	TLS, хадгалалтын шифрлэлт, ADP идэвхтэй үед өргөтгөсөн E2EE
Microsoft OneDrive	OneDrive/SharePoint нэгдсэн хадгалалтын архитектур	Geo-replication, version history, recycle bin, сэргээх боломж	TLS, BitLocker, per-file AES-256, Key Store тусгаарлалт

Хүснэгт 1.1: Нийтийн үүлэн платформуудын архитектур ба шийдлүүдийн харьцуулалт

Дээрх шинжилгээнээс харахад гурван платформын нийтлэг чиглэл нь control plane ба data plane-ийг салгах, тархмал хадгалалт ашиглах, тасалдалд тэсвэртэй replication/failover хэрэгжүүлэх, мөн шифрлэлтийн олон давхар хамгаалалт нэвтрүүлэхэд төвлөрч байна. Харин хэрэглэгч өөрийн өгөгдлийн түлхүүр болон дэд бүтцэд бүрэн хяналт тавих боломж нь үйлчилгээ тус бүр дээр ялгаатай тул self-hosted хувилбарын хэрэгцээ хэвээр байна.

Зураг 1.5: Өргөн ашиглагдаж буй үүлэн хадгалалтын үйлчилгээнүүд

1.2.3 Хувийн үүлэн хадгалалтын системүүд

Self-hosted cloud системүүдийн түгээмэл жишээнүүд:

1. Nextcloud (opensource)
2. ownCloud (opensource)

3. Seafile (opensource)

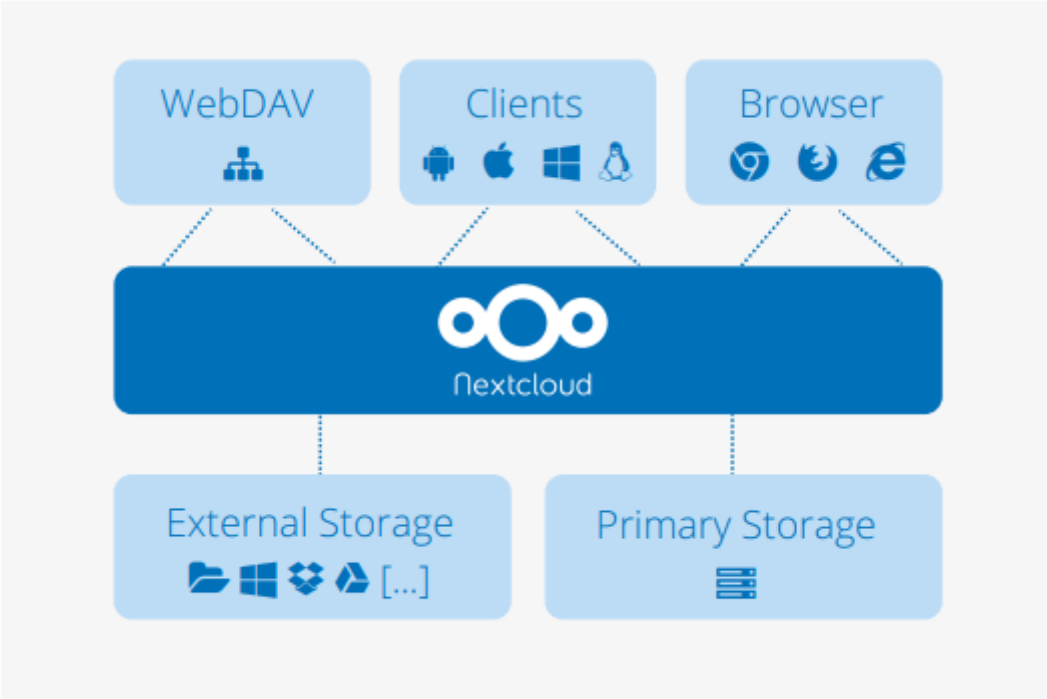
Self-hosted cloud системүүд нь сервер төвтэй архитектур дээр суурилан хэрэглэгчийн өгөгдлийг өөрийн хяналт дор хадгалах боломж олгодог. Эдгээр платформ нь байгууллагын түвшний эрхийн удирдлага, дотоод бодлогын дагуу нэвтрэлт болон хадгалалтын хяналт хийх давуу талтай боловч сүлжээ, серверийн тохиргоо болон тогтмол арчилгааны нэмэлт мэдлэг шаарддаг.

Зураг 1.6: Хувийн сервер дээр ашиглах боломжтой үүлэн хадгалалтын платформууд

Nextcloud

Nextcloud нь self-hosted cloud storage системүүдийн өргөн хэрэглэгддэг төлөөлөл бөгөөд файл хадгалалт, синхрончлол, хуваалцах, хамтын ажиллагааны үйлдлүүдийг нэг платформд нэгтгэдэг. Архитектурын хувьд веб сервер, өгөгдлийн сан, файлын хадгалалтын давхарга, background job зэрэг үндсэн бүрэлдэхүүн дээр ажилладаг бөгөөд Universal File Access [14] давхаргаар олон төрлийн storage эх үүсвэрийг нэг интерфэйс дор уялдуулах боломжтой.

Мөн хэрэглэгч болон багийн түвшинд эрхийн удирдлага, share link, activity log, өргөтгөлийн экосистем зэрэг боломжууд нь Nextcloud-ийг байгууллагын хэрэглээнд уян хатан болгодог. Гэхдээ бодит орчинд тогтвортой ажиллуулахын тулд өгөгдлийн сан, кэш, HTTPS сертификат, хадгалалтын volume, нөөцлөлтийн бодлого зэрэг хэсгүүдийг зөв уялдуулах шаардлагатай байдаг нь энгийн хэрэглэгчдэд суулгалт болон арчилгааны ачаалал нэмэгдүүлдэг.



Зураг 1.7: Nextcloud системийн өгөгдлийн давхаргын нэгдсэн шийдэл

Nextcloud-ийг Docker орчинд ажиллуулсан туршилтын хэсэг

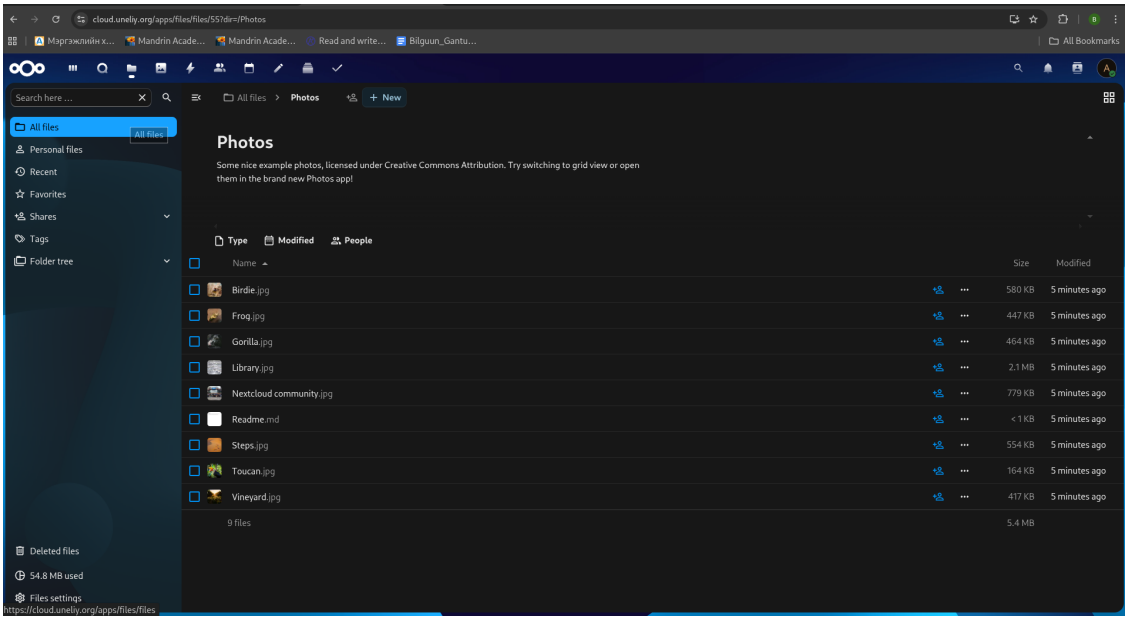
Энэ дэд хэсэгт Nextcloud-ийг Docker орчинд ажиллуулсан туршилтын явц, ашигласан deployment бүтэц, тулгарсан хүндрэлүүд, тэдгээрийг шийдсэн арга барил болон практик үнэлгээг танилцуулна.

Nextcloud-ийг ажиллуулахдаа AIO бусад шаардалагатай docker container-уудаа өөрөө асаагаад удирддаг ажиллагааны төрлийг сонгон ажиллуулсан. Мөн ажиллуулах явцаа юу ч хийгээгүй байх үеийн RAM ашиглалтыг нь асахын өмнө болон ассаны дараах гэсэн байдлаар харьцуулалт гаргасан. Нийт үр дүнгээр nextcloud-aio-apache, nextcloud-aio-nextcloud, nextcloud-aio-imaginary, nextcloud-aio-redis, nextcloud-aio-database, nextcloud-aio-whiteboard, nextcloud-aio-notify-push, nextcloud-aio-talk, nextcloud-aio-mastercontainer гэсэн нийт 9 docker container асаж байсан бөгөөд нийт нийлбэр RAM зарцуулалт нь 1006.6 MiB буюу ойролцоогоор 1 GB орчим RAM-ийг ямар ч үйлдэл хийгээгүй байхад ашиглаж байгаа юм. Nextcloud нь дундаж хэрэглээнд 4GB ram-тай сервер ашиглахыг зөвлөдөг[15]. Дараах зурагт нийт ассан container-уудын RAM болон CPU ашиглалтыг харуулав.

```
CONTAINER ID   NAME                        CPU %     MEM USAGE / LIMIT     MEM %
d92b16684685   nextcloud-aio-apache        0.03%     79.58MiB / 46.88GiB   0.17%
642a09d6396b   nextcloud-aio-nextcloud     0.03%     281.7MiB / 46.88GiB   0.59%
b02796f21b65   nextcloud-aio-imaginary     0.01%     225.6MiB / 46.88GiB   0.47%
ab9e15449795   nextcloud-aio-redis         0.27%     9.906MiB / 46.88GiB   0.02%
d53d07c76423   nextcloud-aio-database      3.91%     68.51MiB / 46.88GiB   0.14%
17fee3931d78   nextcloud-aio-whiteboard    0.01%     132.7MiB / 46.88GiB   0.28%
7cc917e82e7a   nextcloud-aio-notify-push   0.04%     5.34MiB / 46.88GiB    0.01%
134450ec6ec2   nextcloud-aio-talk          0.83%     126.9MiB / 46.88GiB   0.26%
07a7c84ccab0   nextcloud-aio-mastercontainer 0.42%   76.36MiB / 46.88GiB   0.16%
```

Зураг 1.8: Nextcloud-ийг Docker ашиглан асаасны дараа нийт RAM-ийн хэрэглээ

Мөн nextcloud асаахад cloudflare tunnel технелогийг ашигласан бөгөөд өөрийн худалдаж авсан domain дээр subdomain ашиглан nextcloud-ийг ажиллуулсан байгаа. Дараах зурагт nextcloud-ийн хэрэглэгчийн интерфэйсийг харуулав.



Зураг 1.9: Nextcloud-ийн веб хэрэглэгчийн интерфэйс

1.2.4 Peer-to-Peer өгөгдөл синхрончлолын системүүд

Peer-to-peer (P2P) зарчимд суурилсан өгөгдөл синхрончлолын системүүд:

1. Syncthing (opensource)
2. Resilio Sync

Эдгээр системүүд нь төхөөрөмж бүрийг давтагдашгүй таних тэмдэг (device identity)-ээр тодорхойлж, төв сервер шаардахгүйгээр хооронд нь шууд холбох боломж олгодог. Syncthing-ийн хувьд Local Discovery, Global Discovery, Relay Protocol зэрэг механизмуудыг [16] ашиглан сүлжээний янз бүрийн орчинд холболтыг үргэлжлүүлэх боломжтой. Мөн өгөгдөл дамжуулалт нь E2EE зарчмаар хамгаалагдах боломжтой [17].

Гэхдээ P2P системүүд нь төхөөрөмжүүдийн хооронд өгөгдлийг харилцан хуулбарлах (replication) зарчимд тулгуурладаг тул өгөгдөл нэг төв байршилд хадгалагдах уламжлалт хувийн cloud storage загвараас ялгаатай. Иймээс тэдгээрийн хэрэглээний зорилго бүрэн адил биш боловч холболт, илрүүлэлт, аюулгүй дамжуулалтын технологийн хувьд судлан хэрэгжүүлэхэд өндөр ач холбогдолтой.

1.2.5 Харьцуулсан шинжилгээ

Шалгуур	**Нийтийн cloud**	**Self-hosted cloud**	**P2P sync**
Өгөгдлийн өмчлөл ба хяналт	Гуравдагч талын дэд бүтэцт төвлөрсөн	Хэрэглэгч өөрийн сервер дээр бүрэн хяналттай	Төхөөрөмж хооронд тархмал хяналт
Суулгалт ба арчилгаа	Харьцангуй энгийн	Тохиргоо, арчилгаа өндөр шаардлагатай	Peer холболтын орчноос хамааран дунд төвөгтэй
Аюулгүй байдлын хяналт	Үйлчилгээ үзүүлэгчийн бодлогоос ихээхэн хамаарна	Локал бодлогоо өөрөө удирдах боломжтой	Дамжуулалтын хамгаалалт сайн боловч төвлөрсөн бодлого сул
Сүлжээний загвар	Төв сервер төвтэй	Хэрэглэгчийн сервер төвтэй	Discovery/relay төвтэй P2P загвар
Өдөр тутмын хэрэглээний UX	Хамгийн энгийн, автоматжуулалт өндөр	Боломж өргөн ч техник мэдлэг шаарддаг	Sync хэрэглээнд хүчтэй, cloud-like UX харьцангуй хязгаарлагдмал

Хүснэгт 1.2: Ижил төст системүүдийн харьцуулсан шинжилгээ

Харьцуулсан шинжилгээнээс үзэхэд өгөгдөлдөө бүрэн хяналт тогтоох зорилгод self-hosted загвар хамгийн нийцтэй боловч хэрэглэхэд энгийн байдал болон ажиллагааны

төвөгшлийг бууруулах нэмэлт шийдэл шаардлагатай байна. Иймээс энэхүү дипломын ажлын гол чиглэл нь self-hosted зарчмыг хадгалсан атлаа энгийн хэрэглэгчид ашиглахад ойлгомжтой архитектур, интерфэйс, эрхийн удирдлагыг боловсруулахад төвлөрч байна.

1.3 Ашиглах технологиуд

Сервер талын програм хангамжийг Rust програмчлалын хэл дээр хэрэгжүүлнэ. Rust нь Mozilla-аас эхлүүлэн хөгжүүлсэн хөрвүүлэгддэг системийн програмчлалын хэл бөгөөд санах ойн аюулгүй байдал (memory safety), өндөр гүйцэтгэл, найдвартай байдлыг зэрэг хангах зорилготой [18]. C болон C++ хэлүүдийн нэгэн адил машин код руу шууд хөрвүүлэгддэг тул interpreted хэлнүүд болох Python, JavaScript зэрэгтэй харьцуулахад тооцооллын нөөцийг үр ашигтай ашиглах боломжтой.

Rust хэл нь ownership, borrowing, lifetime зэрэг механизмуудаар дамжуулан санах ойтой холбоотой нийтлэг алдаануудыг compile хийх шатанд илрүүлэх боломж олгодог [18]. Мөн нэг бинар файл хэлбэрээр серверийн програмыг хөрвүүлэх боломжтой тул суулгалт болон байршуулалтын үеийн хамаарлыг багасгаж, нэвтрүүлэлтийн процессыг хялбарчилна.

Мөн Rust хэл нь async/await болон олон урсгалт гүйцэтгэлийг (multithreading) дэмждэг тул сервер олон хэрэглэгчийн хүсэлтийг зэрэг боловсруулж, өндөр ачаалын үед тогтвортой ажиллах боломжийг олгодог. Иймээс энэхүү дипломын ажлын сервер талын хэрэгжүүлэлтэд Rust нь гүйцэтгэл болон найдвартай байдлын хувьд тохиромжтой сонголт болно.

Хэрэглэгчийн талын аппликейшнийг хоёр төрлөөр хөгжүүлнэ. Үүнд iOS үйлдлийн системд зориулсан native аппликейшн болон веб аппликейшн орно.

iOS аппликейшнийг Swift програмчлалын хэл болон SwiftUI framework ашиглан хөгжүүлнэ. Swift нь Apple компанийн хөгжүүлсэн орчин үеийн програмчлалын хэл бөгөөд iOS платформд зориулсан өндөр гүйцэтгэлтэй, найдвартай програм хөгжүүлэхэд өргөн хэрэглэгддэг.

Веб аппликейшнийг Angular framework ашиглан хөгжүүлнэ. Angular нь TypeScript дээр суурилсан орчин үеийн веб framework бөгөөд Single Page Application (SPA) архитектурыг хэрэгжүүлэх боломжийг олгодог. Энэ нь хэрэглэгч веб хөтөч ашиглан сервер дээр хадгалагдсан файлуудыг харах, удирдах боломжийг бүрдүүлнэ.

1.4 Бүлгийн дүгнэлт

Энэ бүлэгт үүлэн хадгалалтын системийн ерөнхий ойлголт, хөгжлийн чиг хандлага, ижил төст платформуудын онцлогийг судалж, нийтийн cloud, self-hosted cloud, P2P sync гэсэн үндсэн хандлагуудыг харьцуулан шинжиллээ. Судалгааны үр дүнд self-hosted загвар нь өгөгдлийн өмчлөл болон хяналтын шаардлагад хамгийн нийцтэй боловч суулгалт, арчилгааны төвөгшлийг бууруулах хэрэгцээ өндөр болохыг тогтоов. Мөн Rust, Swift/SwiftUI, Angular зэрэг технологийн сонголтын үндэслэлийг тодорхойлж, дараагийн бүлгүүдийн шаардлагын шинжилгээ болон системийн зохиомжийн суурийг бүрдүүлэв.

БҮЛЭГ 2

Хувийн үүлэн хадгалалтын системийн шинжилгээ

2.1 Системийн үйл ажиллагааны тухай дэлгэрэнгүй

Хэрэглэгч үндсэн серверийн программыг өөрийн хүссэн серверийн компьютер дээр ажиллуулах бөгөөд сүлжээний хандалтын асуудлыг өөрөө шийдвэрлэнэ. Серверийн програмыг ажиллуулсны дараа хэрэглэгч системийн admin interface-д нэвтэрч анхны администратор (admin) хэрэглэгчийг үүсгэнэ. Мөн энэ үед системийн үндсэн үүлэн хадгалалтын хавтсыг (storage directory) зааж өгнө.

Анхны тохиргоо хийгдсэний дараа хэрэглэгч веб аппликейшн эсвэл iOS гар утасны аппликейшн ашиглан системд нэвтрэх боломжтой болно. Администратор хэрэглэгч нь системд шинэ хэрэглэгч үүсгэх, хэрэглэгчийн мэдээллийг удирдах, мөн хэрэглэгчдэд тодорхой хавтас болон файл дээр read эсвэл edit эрх олгох боломжтой байна.

Администратор хэрэглэгч нь системийн хадгалалтын орчинд дурын хавтас болон файлыг үүсгэх, засах, устгах, мөн файлуудыг download болон upload хийх бүрэн эрхтэй байна.

Энгийн хэрэглэгч нь администратороос олгосон эрхийн хүрээнд системд хадгалагдсан файлууд болон хавтаснуудтай ажиллах боломжтой бөгөөд файлыг харах, татаж авах, эсвэл засварлах зэрэг үйлдлүүдийг зөвшөөрөгдсөн хэмжээнд гүйцэтгэнэ.

2.2 Функциональ шаардлага

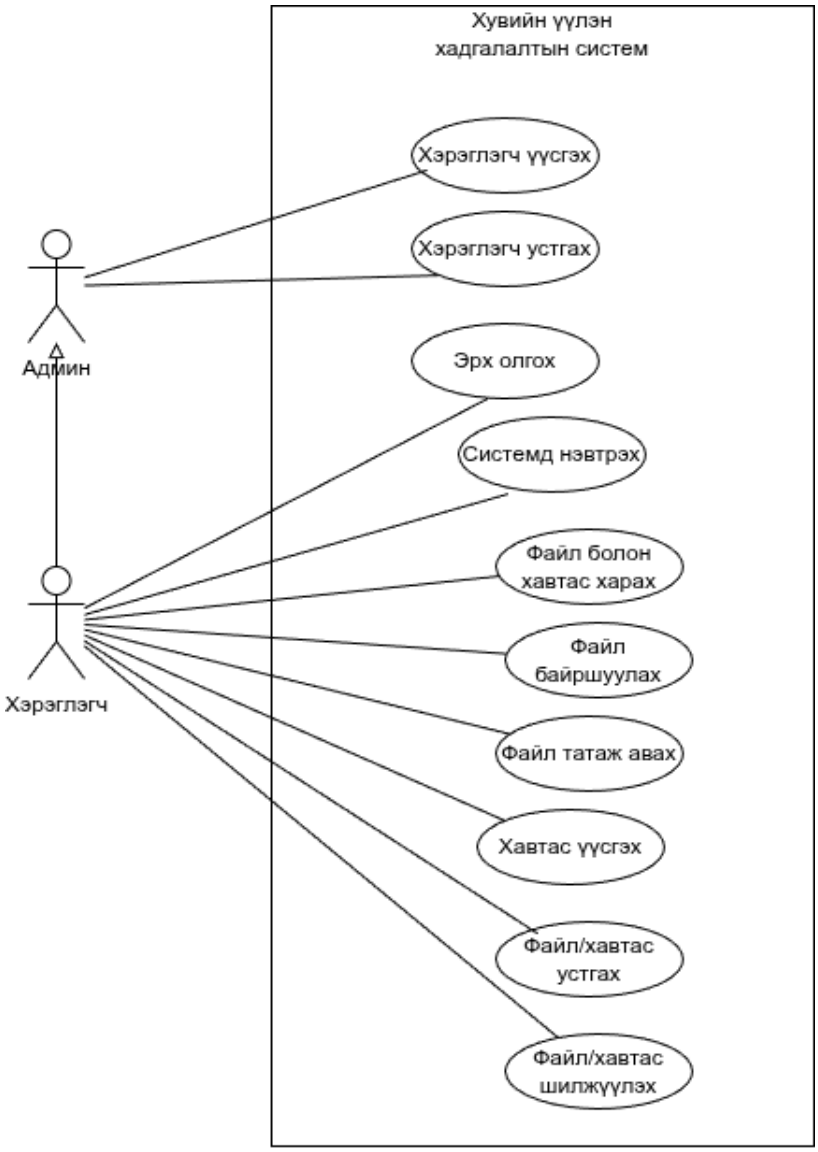
1. Серверийг анх ажиллуулах үед анхны админ хэрэглэгчийг үүсгэх боломжтой байх.
2. Үүлэн хадгалалтад ашиглах үндсэн хадгалалтын хавтсыг (storage folder) сонгон тохируулах боломжтой байх.
3. Хэрэглэгч веб аппликейшн болон iOS гар утасны аппликейшн ашиглан системд нэвтрэх боломжтой байх.
4. Админ шинэ хэрэглэгч үүсгэх боломжтой байх.
5. Админ хэрэглэгчийн нууц үгийг өөрчлөх болон хэрэглэгчийг устгах боломжтой байх.
6. Админ хэрэглэгчдэд тодорхой файл болон хавтас дээр read эсвэл edit эрх олгох боломжтой байх.
7. Хэрэглэгч өөрийн edit эрхтэй файл болон хавтас дээр бусад хэрэглэгчдэд эрх олгох боломжтой байх.

8. Хэрэглэгч системд хадгалагдсан файл болон хавтасны бүтцийг харах боломжтой байх.
9. Хэрэглэгч сервер рүү файл байршуулах (upload) боломжтой байх.
10. Хэрэглэгч системд хадгалагдсан файлыг татаж авах (download) боломжтой байх.
11. Хэрэглэгч шинэ хавтас үүсгэх боломжтой байх.
12. Хэрэглэгч файл болон хавтсын нэрийг өөрчлөх боломжтой байх.
13. Хэрэглэгч файл болон хавтсыг устгах боломжтой байх.
14. Хэрэглэгч файл болон хавтсыг өөр хавтас руу шилжүүлэх боломжтой байх.
15. Хэрэглэгч дараах төрлийн файлуудыг preview хийж харах боломжтой байх. Зураг (jpg, png, gif, webp), Text (txt, json, csv, md), Documents (pdf), Audio (mp3), Video (mp4)

2.2.1 Usecase диаграмм

Дараах юзкейс диаграммд системийн үндсэн 2 хэрэглэгч болох admin болон энгийн хэрэглэгч гэсэн 2 оролцогчийг дүрслэв. Admin хэрэглэгч нь энгийн хэрэглэгчээс удамшиж байгаа ба энгийн хэрэглэгчийн бүх usecase-ийг хийх чадамжтай юм.

Системийн эхний setup хийх хэсэг буюу admin хэрэглэгчийг үүсгэх болон үндсэн хавтасыг сонгох хэсгийг энэхүү usecase диаграмм дээр оруулаагүй болно.



Зураг 2.1: Usecase диаграм

2.2.2 Юзкейсийн тодорхойлолт

Хэрэглэгч үүсгэх (UC-001)

Usecase ID	UC-001
Usecase name	Хэрэглэгч үүсгэх
Actor(s)	Админ
Description	Админ хэрэглэгч үүлэн хадгалалтын системд шинэ хэрэглэгч үүсгэх боломжтой байна.
Trigger	Үүлэн хадгалалтын системийг өөр хүмүүстэй хамт ашиглах шаардлага гарсан тохиолдолд
Preconditions	• Админ системд нэвтэрсэн байх.
Postconditions	• Шинэ хэрэглэгч үүссэн бөгөөд системд нэвтрэх нэр болон нууц үгээ ашиглан нэвтрэх боломжтой байх.
Main success scenario	1. Админ хэрэглэгчийн менежмент хэсэг рүү орно. 2. Шинэ хэрэглэгч үүсгэх товчийг дарна. 3. Хэрэглэгчийн нэвтрэх нэр болон ашиглах нууц үгийг оруулна. 4. Үүсгэх товчийг дарна. 5. Хэрэглэгч амжилттай үүсэж хэрэглэгчийг жагсаалт дээр харагдана.
Alternative flows	
Exception flows	Хэрэглэгчийн нэр давхцвал алдаа гарна.
Acceptance criteria	Шинэ хэрэглэгч амжилттай бүртгэгдсэн байх.
Special requirements	

Хүснэгт 2.1: UC-001 usecase-ийн тодорхойлолт

Хэрэглэгч устгах (UC-002)

Usecase ID	UC-002
Usecase name	Хэрэглэгч устгах
Actor(s)	Админ
Description	Админ систем дээр бүртгэлтэй байгаа хэрэглэгчдийг устгах боломжтой байна.
Trigger	Устгах шаардлагатай хэрэглэгч гарсан.
Preconditions	• Админ системд нэвтэрсэн байх.
Postconditions	• Хэрэглэгч амжилттай устгагдсан байх. • Хэрэглэгчийн мэдээлэл өгөгдлийн сангаас устсан байх.
Main success scenario	1. Админ хэрэглэгчийн менежмент хэсэг рүү орно. 2. Хэрэглэгчийн жагсаалтаас устгах хэрэглэгчээ сонгож устгах товч дарна. 3. Баталгаажуулах цонх гарч ирж баталгаажуулна. 4. Хэрэглэгчийг устгана.
Alternative flows	
Exception flows	
Acceptance criteria	Хэрэглэгч устсан байх. Тухайн хэрэглэгчээр нэвтрэх боломжгүй болсон байх.
Special requirements	Хүсэлт аюулгүй сувгаар дамжих.

Хүснэгт 2.2: UC-002 usecase-ийн тодорхойлолт

Эрх олгох (UC-003)

Usecase ID	UC-003
Usecase name	Эрх олгох
Actor(s)	Админ, Хэрэглэгч
Description	Хэрэглэгч өөрийн эрхтэй хавтас болон файл дээр бусад хэрэглэгч нарт эрх олгох боломжтой. Админ өөрийн хүссэн хавтас болон файл дээр хүссэн хэрэглэгчдээ эрх өгөх боломжтой.
Trigger	Бусад хэрэглэгч нарт ямар нэгэн хавтас болон файл хуваалцах шаардлага гарсан тохиолдолд
Preconditions	• Тухайн хэрэглэгч нэвтэрсэн байх. • Хуваалцах гэж буй хавтас болон файл тухайн хэрэглэгчийнх байх.
Postconditions	• Эрхийг авч буй хэрэглэгч дээр эрх нэмэгдсэн байх.
Main success scenario	1. Хэрэглэгч үүлэн хадгалалт дотроо эрх олгох гэж буй файл болон хавтсан дээрээ очно. 2. Хуваалцах товч дээр дарна. 3. Хэрэглэгч сонгоод баталгаажуулах товч дарна.
Alternative flows	
Exception flows	
Acceptance criteria	Хэрэглэгч дээр эрх үүссэн байх.
Special requirements	Хэрэглэгч тухайн файл эсвэл хавтасны эзэн биш бол хуваалцах товч харагдах ёсгүй. Мөн эзэн мөн эсэх шалгалтыг сервер талд гүйцэтгэнэ.

Хүснэгт 2.3: UC-003 usecase-ийн тодорхойлолт

Системд нэвтрэх (UC-004)

Usecase ID	UC-004
Usecase name	Системд нэвтрэх
Actor(s)	Админ, хэрэглэгч
Description	Үүлэн хадгалалтын үндсэн систем рүү нэвтрэх
Trigger	Хувийн үүлэн хадгалалтын системийг ашиглах шаардлага гарсан.
Preconditions	• Тухайн нэвтрэх гэж буй хэрэглэгчийн мэдээлэл системд бүртгэгдсэн байх.
Postconditions	• Системд амжилттай нэвтэрч өөрт хамааралтай хавтас болон файлуудыг харах
Main success scenario	1. Хэрэглэгч нэвтрэх хуудсыг нээнэ. 2. Нэвтрэх нэр болон нууц үгийг оруулаад нэвтрэх товчийг дарна. 3. Амжилттай тохиолдолд хэрэглэгчийг нэвтрүүлнэ. 4. Хэрэглэгч үндсэн хуудас руу үсэрнэ.
Alternative flows	
Exception flows	Нэвтрэх нэр болон нууц үг буруу бол алдааны мессеж харуулна.
Acceptance criteria	Хэрэглэгч системд амжилттай нэвтэрсэн байна.
Special requirements	

Хүснэгт 2.4: UC-004 usecase-ийн тодорхойлолт

Файл болон хавтас харах (UC-005)

Usecase ID	UC-005
Usecase name	Файл болон хавтас харах (Browse хийх)
Actor(s)	Хэрэглэгч
Description	Хадгалагдаж буй файлууд болон хавтасаар зочилж, сонирхох
Trigger	Файл эсвэл хавтас хайх шаардлага гарсан эсвэл ямар файл болон хавтаснууд байгааг сонирхсон.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх.
Postconditions	•
Main success scenario	1. Хэрэглэгч хүссэн хавтас болон файлуудаа харна.
Alternative flows	Хэрэглэгч хайлтын хэсгээс файл болон хавтсыг нэрээр хайх эсвэл файлын төрлөөр хайх боломжтой.
Exception flows	
Acceptance criteria	Хэрэглэгч эрхтэй файлууд болон хавтсуудаа үзсэн.
Special requirements	Хэрэглэгчид зөвхөн эрхтэй хавтас болон файлуудыг харуулна.

Хүснэгт 2.5: UC-005 usecase-ийн тодорхойлолт

Файл байршуулах (UC-006)

Usecase ID	UC-006
Usecase name	Файл байршуулах
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч өөрийн owner болон edit эрхтэй хавтас дотор файл байршуулах боломжтой.
Trigger	Хэрэглэгч өөрийн үүлэн хадгалалтын төхөөрөмж дээр файл байршуулах шаардлага гарах.
Preconditions	• Хэрэглэгч нэвтэрсэн байх.
Postconditions	• Файл системийн файл системд байршсан байх.
Main success scenario	1. Хэрэглэгч үндсэн хуудас дээрх файл системээр явж файл байршуулах хавтас руугаа орно. 2. Файл байршуулах товчийг сонгоно. 3. Файл сонгох цонх гарч ирж хэрэглэгч нэг болон түүнээс их файл сонгоно. 4. Файлуудыг upload хийгдэж буй явц харагдана.
Alternative flows	
Exception flows	Файлыг upload хийх явцад сүлжээ тасрах магадлалтай.
Acceptance criteria	Тухайн файл сервер дээр хадгалагдах
Special requirements	

Хүснэгт 2.6: UC-006 usecase-ийн тодорхойлолт

Файл татаж авах (UC-007)

Usecase ID	UC-007
Usecase name	Файл татаж авах
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч өөрийн read, edit, owner эрхтэй файлыг татах
Trigger	Хэрэглэгчид файлыг татах шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх.
Postconditions	• Файл хэрэглэгчийн төхөөрөмж дээр татагдсан байх
Main success scenario	1. Хэрэглэгч үндсэн хуудас дээрх файл системээр явж татахыг хүссэн файлынхаа хавтас руу орно. 2. Татах товчийг дарна. 3. Файл татах явц харагдана.
Alternative flows	
Exception flows	Файлыг татах явцад сүлжээ тасрах магадлалтай
Acceptance criteria	Файл хэрэглэгчийн төхөөрөмж дээр татагдсан байх
Special requirements	

Хүснэгт 2.7: UC-007 usecase-ийн тодорхойлолт

Хавтас үүсгэх (UC-008)

Usecase ID	UC-008
Usecase name	Хавтас үүсгэх
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч шинэ хавтас үүсгэж өөрийн файлуудыг зохион байгуулна.
Trigger	Хэрэглэгч шинэ хавтас үүсгэх шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх. • Хэрэглэгч хавтас үүсгэх эрхтэй байх.
Postconditions	• Шинэ хавтас системд амжилттай үүссэн байна.
Main success scenario	1. Хэрэглэгч файл системийн интерфэйс дээр байрлах хавтас үүсгэх товчийг дарна. 2. Шинэ хавтасны нэрийг оруулна. 3. Систем шинэ хавтсыг үүсгэж харуулна.
Alternative flows	1. Хэрэглэгч хавтасны нэрийг өөрчлөх боломжтой.
Exception flows	Ижил нэртэй хавтас аль хэдийн үүссэн байж болно.
Acceptance criteria	Шинэ хавтас файл системд харагдаж байх.
Special requirements	

Хүснэгт 2.8: UC-008 usecase-ийн тодорхойлолт

Файл/хавтас устгах (UC-009)

Usecase ID	UC-009
Usecase name	Файл/хавтас устгах
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч өөрийн эрхтэй файл эсвэл хавтсыг системээс устгана.
Trigger	Хэрэглэгч файл эсвэл хавтас устгах шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх. • Хэрэглэгч тухайн файл эсвэл хавтсыг устгах эрхтэй байх.
Postconditions	• Файл эсвэл хавтас системээс устсан байна.
Main success scenario	1. Хэрэглэгч устгахыг хүссэн файл эсвэл хавтсыг сонгоно. 2. Устгах товчийг дарна. 3. Систем баталгаажуулах цонх харуулна. 4. Хэрэглэгч баталгаажуулна. 5. Систем файл эсвэл хавтсыг устгана.
Alternative flows	1. Хэрэглэгч устгах үйлдлийг цуцалж болно.
Exception flows	Файл ашиглагдаж байгаа үед устгах боломжгүй байж болно.
Acceptance criteria	Файл эсвэл хавтас системээс бүрэн устсан байх.
Special requirements	

Хүснэгт 2.9: UC-009 usecase-ийн тодорхойлолт

Файл/хавтас шилжүүлэх (UC-010)

Usecase ID	UC-010
Usecase name	Файл/хавтас шилжүүлэх
Actor(s)	Хэрэглэгч
Description	Хэрэглэгч файл эсвэл хавтсыг өөр хавтас руу шилжүүлнэ.
Trigger	Хэрэглэгч файл эсвэл хавтасны байршлыг өөрчлөх шаардлага гарсан.
Preconditions	• Хэрэглэгч системд нэвтэрсэн байх. • Хэрэглэгч тухайн файл эсвэл хавтсыг өөрчлөх эрхтэй байх.
Postconditions	• Файл эсвэл хавтас шинэ байршилд шилжсэн байна.
Main success scenario	1. Хэрэглэгч шилжүүлэх файл эсвэл хавтсыг сонгоно. 2. Шилжүүлэх үйлдлийг сонгоно. 3. Зорилтот хавтсыг сонгоно. 4. Систем файл эсвэл хавтсыг шинэ байршилд шилжүүлнэ.
Alternative flows	1. Хэрэглэгч шилжүүлэх үйлдлийг цуцалж болно.
Exception flows	Зорилтот хавтас олдохгүй байх магадлалтай.
Acceptance criteria	Файл эсвэл хавтас шинэ хавтас дотор харагдаж байх.
Special requirements	

Хүснэгт 2.10: UC-010 usecase-ийн тодорхойлолт

2.3 Функциональ бус шаардлага

1. Серверийн программ нь Linux, Windows, MacOS үйлдлийн системүүд дээр ажиллах боломжтой байх.

2. Хэрэглэгчийн client аппликейшн нь web болон iOS платформ дээр ажиллах боломжтой байх.
3. Web аппликейшн нь Chrome, Firefox, Safari болон Edge браузеруудын сүүлийн хоёр хувилбар дээр зөв ажиллах боломжтой байх.
4. Хэрэглэгч болон серверийн хооронд дамжих бүх өгөгдөл HTTPS (TLS) протокол ашиглан шифрлэгдсэн байх.
5. Хэрэглэгчийн нууц үгийг сервер дээр plaintext хэлбэрээр хадгалахгүй бөгөөд аюулгүй hash алгоритм ашиглан хадгалдаг байх.
6. Систем нь нэг сервер дээр олон хэрэглэгчийн зэрэг хандалтыг дэмжих боломжтой байх.
7. Файл байршуулах болон татах үйлдлийн явцыг хэрэглэгчийн интерфэйс дээр харуулах боломжтой байх.
8. Систем нь хэрэглэгч зөвхөн өөрийн файл болон өөрт олгогдсон эрхтэй өгөгдөлд хандах боломжийг хангах байх.
9. Файл дамжуулах явцад өгөгдлийн бүрэн бүтэн байдлыг шалгах механизм (checksum эсвэл hash) ашигладаг байх.

2.4 Ажлын хүрээ

2.4.1 Scope (Хамрах хүрээ)

Энэхүү төслийн хүрээнд дараах програм хангамжийн бүрэлдэхүүнүүдийг хөгжүүлнэ:

1. **Хувийн үүлэн хадгалалтын үндсэн серверийн программ (Main Server Application):** Үндсэн серверийн программ нь хэрэглэгчийн компьютерт суурилж, өгөгдлийг хадгалах, төхөөрөмж бүртгэх, холболтыг удирдах, файл дамжуулалтыг гүйцэтгэх үүрэгтэй байна.
2. **iOS үйлдлийн системд зориулсан мобайл клиент аппликейшн:** iOS мобайл аппликейшн нь хувийн үүлэн хадгалалтын серверийн үндсэн үйлдлүүдийг хийх боломжтой байна. Нэвтрэх, файл болон хавтас үзэх, файл татах, файл байршуулах гэх мэт дээр оруулсан үйлдлүүдийг хийнэ.
3. **Веб апп:** iOS апп-тай адилхан бүх үйлдлийг хийх боломжтой байна. Үндсэн үйлдлээс гадна. Хамгийн эхний тохигооны алхам буюу админ хэрэглэгч үүсгэх болон үндсэн хавтас сонгох алхамууд нэмэгдэж хийгдэнэ.

2.4.2 Out of Scope (Хийхгүй зүйлс)

1. **Android мобайл аппликейшн:**

Android үйлдлийн системд зориулсан мобайл аппликейшн энэхүү төслийн хүрээнд хөгжүүлэхгүй. Төслийн хүрээнд нэг платформ дээр төвлөрч, native хөгжүүлэлтийг гүнзгий түвшинд хийх зорилгоор iOS платформыг сонгосон болно.

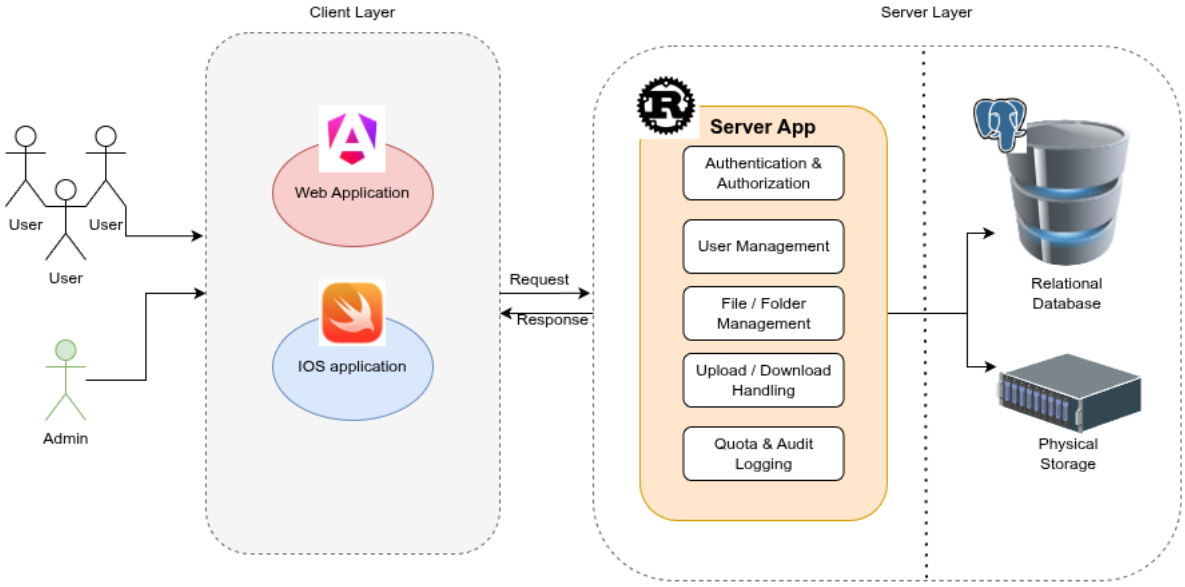
2.5 Бүлгийн дүгнэлт

БҮЛЭГ 3

Хувийн үүлэн хадгалалтын системийн зохиомж ба хөгжүүлэлт

3.1 Системийн архитектур

Дараах зурагт хувийн үүлэн хадгалалтын системийн архитектур диаграммыг харуулав. Энгийн хэрэглэгч болон admin гэсэн 2 төрлийн хэрэглэгч байх агаад бүх хэрэглэгчид веб олон IOS апп ашиглан системтэй харьцана. Веб апп-ийг Angular, IOS апп-ийг Swift технелогиудыг ашиглан хөгжүүлнэ. Тухайн хэрэглэгчийн апп-ууд нь сервертэй HTTPS холболтоор харьцаж зорилтот үйлдлүүдийг гүйцэтгэдэг байна. Сервер апп нь хэрэглэгчдийн нэвтрэлт, баталгаажуулалт, хэрэглэгчийн менежмент, файл менежмент, файл татах болон байршуулах үйлдлүүдийг хийнэ. Системийн нийт өгөгдлийг Relational өгөгдлийн бааз болон серверийн File system-д хадгална. Тухайн үүлэн хадгалалт дээр байрших файлууд болон хавтаснуудын мета өгөгдлүүд нь өгөгдлийн бааз дээр хадгалагдана.



Зураг 3.1: Системийн архитектур

3.2 Зохиомжийн үндсэн компонентууд

Системийн зохиомжид дараах үндсэн компонентуудыг тодорхойлсон.

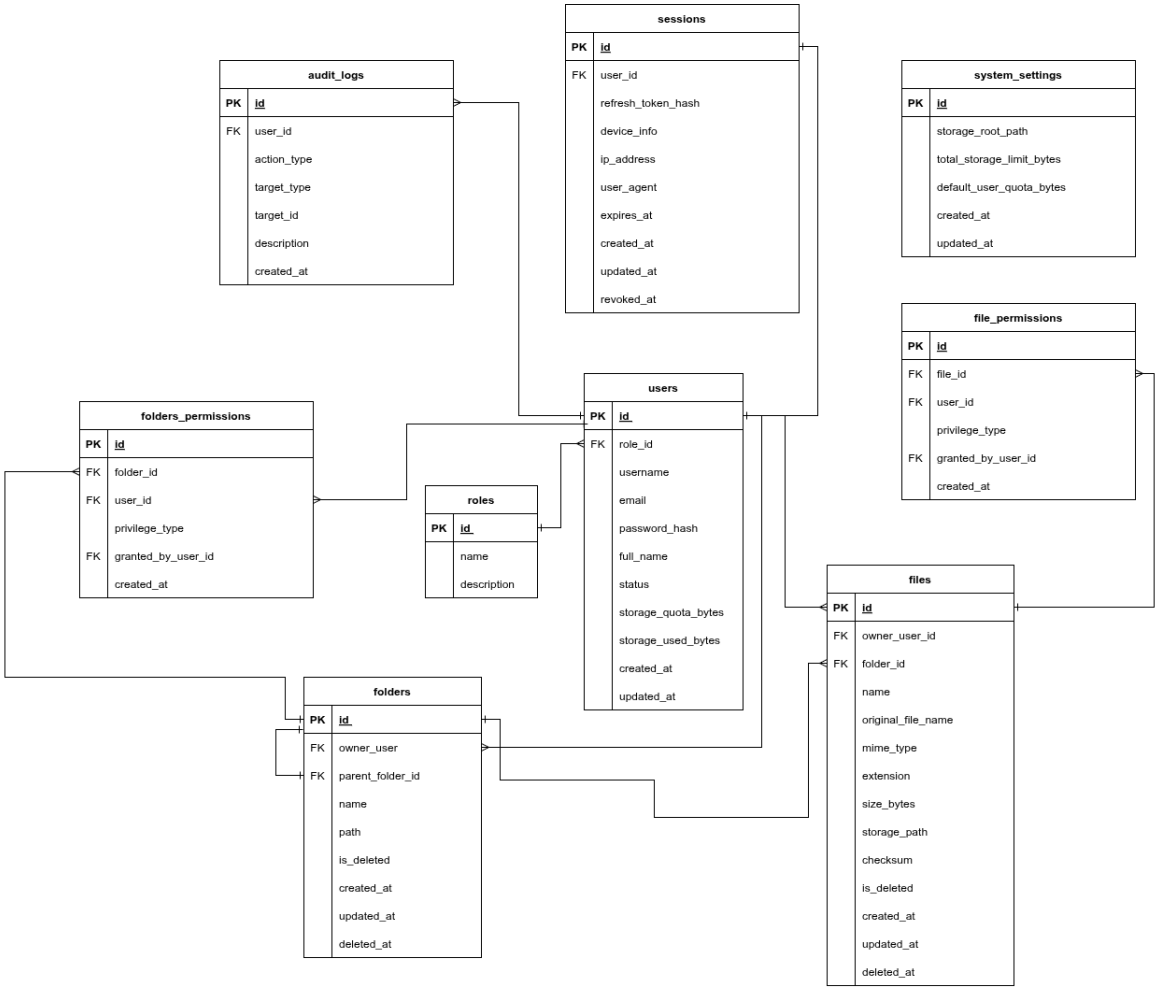
1. **Хэрэглэгчийн удирдлагын модуль** – нэвтрэх, нууц үг шалгах, session/token удирдах.
2. **Файл удирдлагын модуль** – хавтас үүсгэх, файл байршуулах, татах, нэр өөрчлөх, устгах.
3. **Хуваалцах эрхийн модуль** – хэрэглэгч болон бүлгийн түвшинд read/edit эрх олгох.

4. **Storage abstraction** – локал диск дээрх өгөгдөл болон өгөгдлийн сангийн мета мэдээллийг уялдуулах.

3.3 Өгөгдлийн ерөнхий схем

Зурагт үзүүлсэн энэхүү өгөгдлийн сангийн загвар (ERD) нь хувийн үүлэн хадгалалтын системийн үндсэн өгөгдлийн бүтэц болон тэдгээрийн харилцан хамаарлыг тодорхойлж байна. Систем нь хэрэглэгч (*users*) болон үүрэг (*roles*)-ийн удирдлагыг дэмжихийн зэрэгцээ хавтас (*folders*) болон файл (*files*)-ын шаталсан бүтэц бүхий логик хадгалалтыг хэрэгжүүлнэ. Хавтас нь *parent_folder_id* талбараар дамжин өөртэйгээ рекурс хамаарал үүсгэж, мод хэлбэрийн бүтэц бүрдүүлдэг бол файл нь тодорхой хавтсанд хамаарах байдлаар зохион байгуулагдана. Файлын бодит өгөгдөл нь физик хадгалалтын орчинд байрлах бөгөөд *storage_path* талбараар заагдана.

Системд хэрэглэгчийн хандалтын эрхийг удирдах зорилгоор *folder_permissions* болон *file_permissions* хүснэгтүүдийг ашигласан ба эдгээр нь хэрэглэгч бүрт *read* болон *edit* төрлийн эрхийг оноох боломжийг олгоно. Файл болон хавтас бүр нь *owner_user_id* талбараар эзэмшигчийг тодорхойлж, эзэмшигч нь бүрэн хяналттай байх зарчим хэрэгжинэ. Мөн хэрэглэгчийн нэвтрэлтийн сессийг *sessions* хүснэгтээр дамжуулан удирдаж, refresh token-ийн хэш, төхөөрөмжийн мэдээлэл болон хугацааны хязгаарыг хадгална. Үүнээс гадна системийн үйл ажиллагааг бүртгэх зорилгоор *audit_logs* хүснэгтийг ашиглаж, хэрэглэгчийн хийсэн үйлдэл, зорилтот объект болон хугацааг тэмдэглэн хадгална.

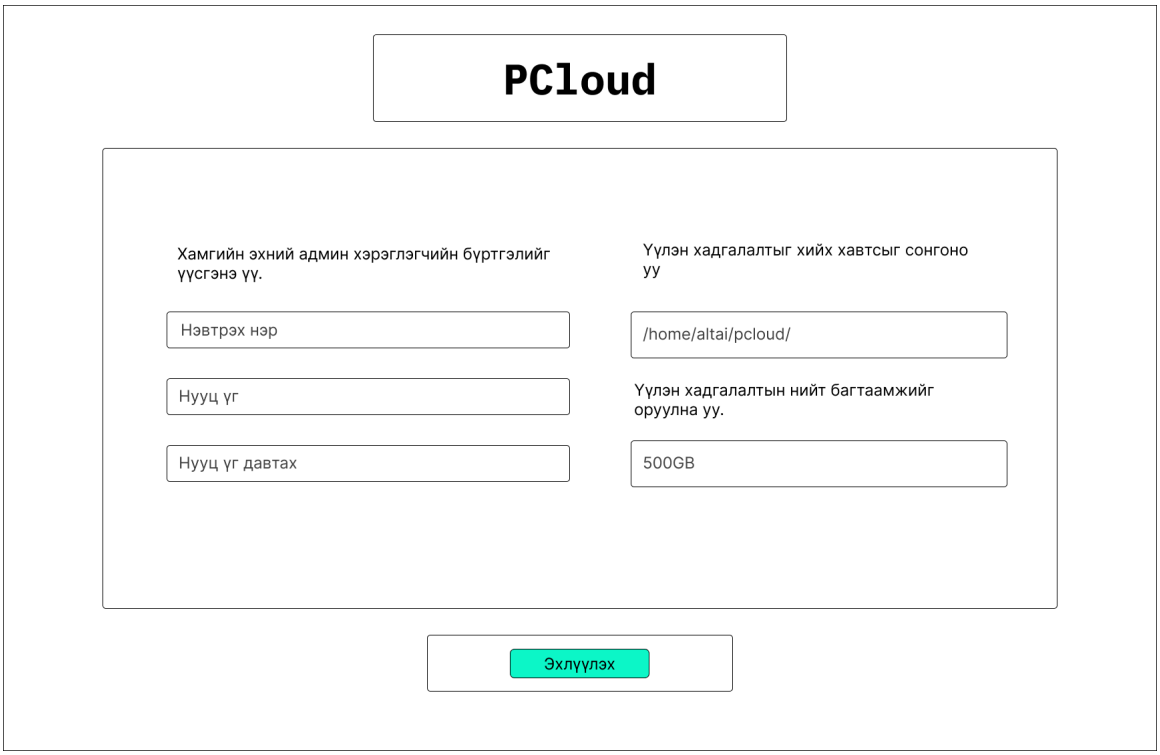


Зураг 3.2: Хувийн үүлэн хадгалалтын системийн ERD

3.4 Системийн прототип

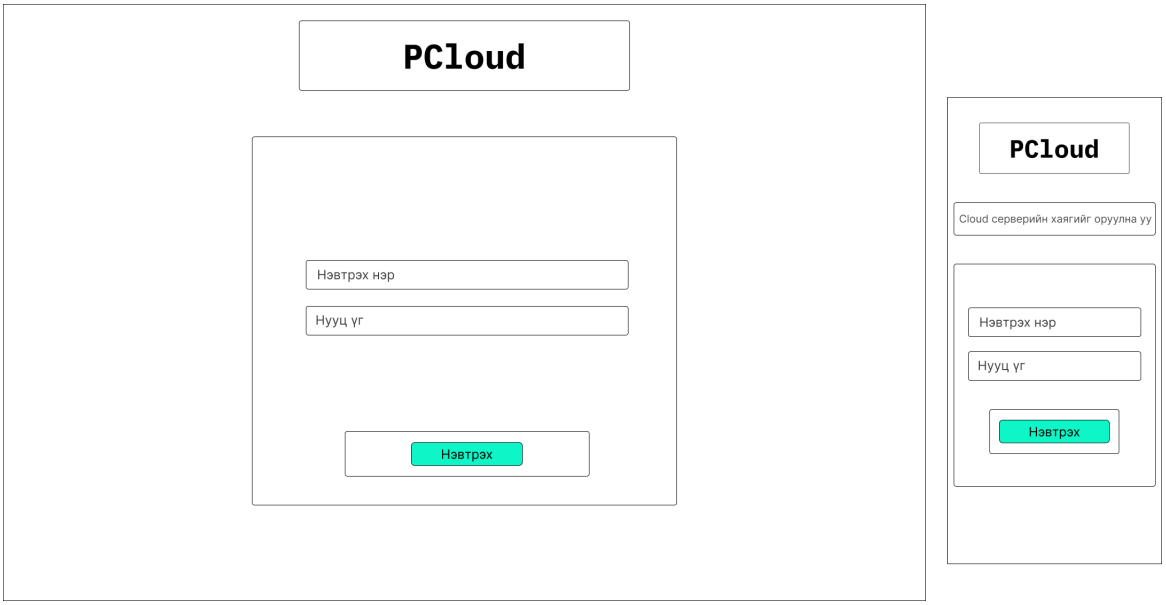
Системийн ерөнхий интерфейсийн прототипийг Figma орчинд бэлтгэсэн. Прототип нь эхний тохиргоо, нэвтрэх хуудас, энгийн хэрэглэгчийн нүүр хуудас, админ хэрэглэгчийн нүүр хуудас гэсэн үндсэн урсгалуудыг хамарсан.

Системийг анх ажиллуулах үед админ хэрэглэгч болон storage folder-ийн үндсэн тохиргоог авах дэлгэц нээгдэнэ. Энэ алхам нь self-hosted системийн хамгийн чухал эхний шат бөгөөд өгөгдөл хаана хадгалагдахыг тодорхойлно.



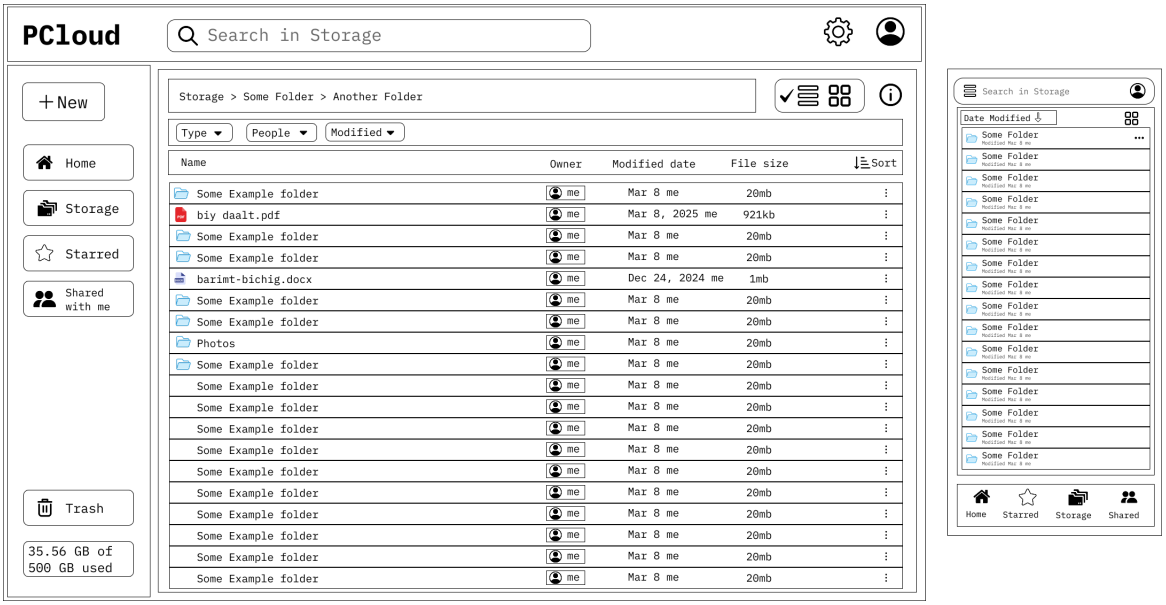
Зураг 3.3: Анхны тохиргооны дэлгэц

Нэвтрэх дэлгэц нь хэрэглэгчийн нэр, нууц үг, шаардлагатай үед серверийн хаяг оруулах талбаруудтай. Энэ загвар нь веб болон мобайл клиентээс нэг ижил backend-т холбогдох боломж олгодог.



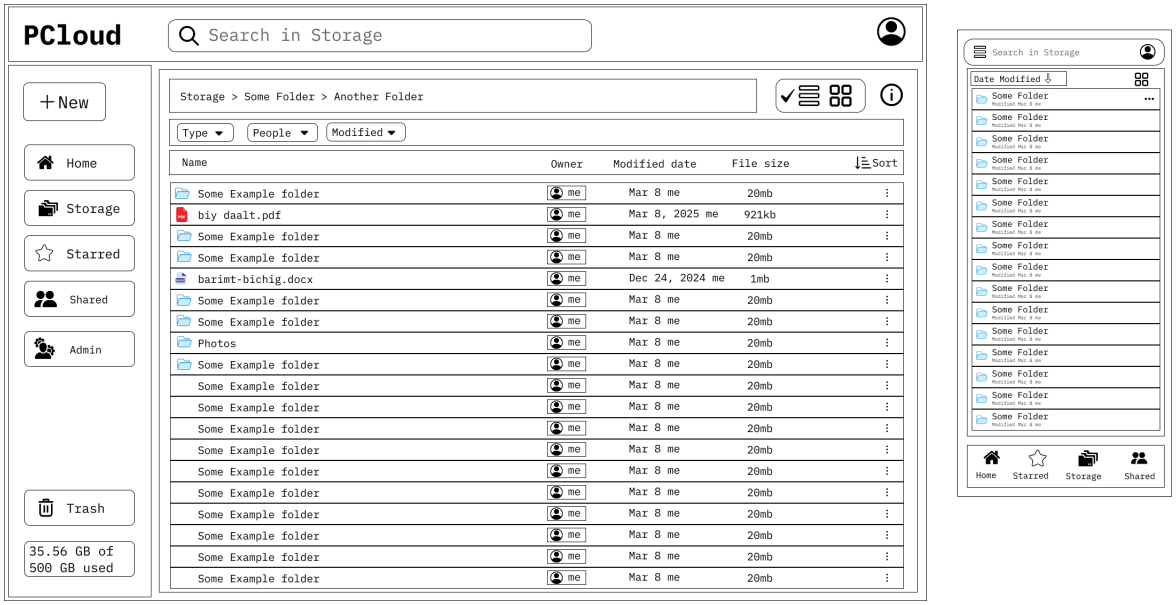
Зураг 3.4: Нэвтрэх дэлгэц

Энгийн хэрэглэгчийн нүүр хуудсан дээр файл болон хавтсын бүтэц харагдаж, upload/download, rename, delete, create folder зэрэг үндсэн үйлдлүүдийг хийх боломжтой. Мөн хайлт, харагдац өөрчлөх, shared файлууд харах зэрэг функцууд багтсан.



Зураг 3.5: Энгийн хэрэглэгчийн үндсэн хуудас

Админ хэрэглэгчийн хувьд дээрх боломжууд дээр нэмээд хэрэглэгч удирдлага, эрхийн бодлого тохируулах, системийн төлөв хянах цэсүүдийг ашиглах боломжтой.



Зураг 3.6: Админ хэрэглэгчийн нүүр хуудас

3.5 Хөгжүүлсэн системийн интерфейс

Хөгжүүлсэн интерфейс нь прототипийн үндсэн урсгалыг хадгалсан бөгөөд дараах дараалаар ажиллана.

1. Анхны тохиргоогоор админ хэрэглэгч болон хадгалалтын үндсэн хавтас үүсгэнэ (Зураг 3.3).
2. Хэрэглэгч серверийн хаяг, хэрэглэгчийн нэр, нууц үгээр нэвтэрнэ (Зураг 3.4).
3. Энгийн хэрэглэгч файлуудаа удирдаж, хуваалцсан агуулгад хандана (Зураг 3.5).
4. Админ хэрэглэгч хэрэглэгч болон системийн ерөнхий тохиргоог удирдана (Зураг 3.6).

3.6 Системд хийгдсэн тест

Системийн баталгаажуулалтыг дараах түвшинд гүйцэтгэсэн.

1. **Функциональ тест**: Нэвтрэх, файл байршуулах, татах, нэр өөрчлөх, устгах, эрх олгох урсгалуудыг usecase тус бүрээр шалгасан.
2. **Эрхийн тест**: Админ болон энгийн хэрэглэгчийн эрх ялгарах, зөвшөөрөлгүй хандалтыг блоклох шалгалт хийсэн.
3. **Интерфейсийн тест**: Веб болон мобайл клиент дээр үндсэн дэлгэцүүдийн хэрэглээний урсгал, алдааны мэдэгдлийг шалгасан.

3.7 Системийн нэвтрүүлэлт

Системийг дотоод туршилтын орчинд self-hosted хэлбэрээр нэвтрүүлж, серверийн локал storage directory-т өгөгдөл хадгалах байдлаар ажиллуулсан. Нэвтрүүлэлтийн үе шатанд хэрэглэгчийн бүртгэл үүсгэх, файл байршуулах, эрх хуваалцах, татах урсгалуудыг бодит хэрэглээний сценариар туршиж, ажиллагааны алдаануудыг засварласан.

3.8 Бүлгийн дүгнэлт

Энэ бүлэгт хувийн үүлэн хадгалалтын системийн архитектур, үндсэн компонентууд, өгөгдлийн зохиомж, прототип болон хэрэгжүүлсэн интерфейсийг тайлбарлалаа. Системийн шийдэл нь хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах self-hosted зарчимд тулгуурласан бөгөөд веб болон мобайл клиентээр дамжуулан нэгдсэн

API-р ажиллахаар зохион байгуулагдсан. Мөн функцийн болон эрхийн түвшний тестүүдээр системийн үндсэн урсгалуудыг баталгаажуулсан.

Ерөнхий дүгнэлт

Энэхүү дипломын ажлын хүрээнд хэрэглэгч өөрийн төхөөрөмж дээр байршуулж ажиллуулах боломжтой хувийн үүлэн хадгалалтын системийн зохиомж, хөгжүүлэлт, туршилтыг хийж гүйцэтгэлээ. Төслийн гол зорилго нь хэрэглэгчийн өгөгдлийг гуравдагч талын серверт бус өөрийн хяналтын орчинд хадгалах, мөн веб болон мобайл клиентээр дамжуулан тухайн өгөгдөлд найдвартай хандах боломж бүрдүүлэхэд оршсон.

Судалгааны шатанд нийтийн болон self-hosted үүлэн хадгалалтын шийдлүүдийг харьцуулж, бага нөөцтэй орчинд хэрэгжих боломжтой архитектурын шаардлагыг тодорхойлсон. Шинжилгээний шатанд системийн функциональ шаардлага, хэрэглэгчийн үүрэг ба эрхийн загвар, гол usecase-уудыг боловсруулсан. Зохиомж, хөгжүүлэлтийн шатанд серверийн API, файл удирдлагын логик, эрхийн хяналт, нэвтрэлтийн урсгал, админ болон энгийн хэрэглэгчийн интерфейсийг хэрэгжүүлсэн.

Үр дүнд нь анхны тохиргоо, хэрэглэгч нэвтрэх, файл байршуулах/татах, хавтас удирдах, хуваалцах эрх олгох зэрэг үндсэн урсгалууд тогтвортой ажиллах хувийн үүлэн хадгалалтын системийг бий болгосон. Цаашид сүлжээний орчинд алсаас найдвартай холбогдох шийдлийг өргөжүүлэх, гүйцэтгэл болон аюулгүй байдлын тестийг илүү өргөн хүрээнд хийх боломжтой.

ХАВСРАЛТ А

Серверийн API-ийн жишээ

A.1 REST endpoint-ууд

Энэ хавсралтад хувийн үүлэн хадгалалтын системийн сервер талын үндсэн API endpoint-уудын жишээг үзүүлэв.

```
POST   /api/auth/login
POST   /api/auth/logout

GET    /api/users/me
POST   /api/users             // admin: create user
DELETE /api/users/{userId}    // admin: delete user

GET    /api/files?path=/      // list files/folders
POST   /api/files/upload      // upload file
GET    /api/files/download/{id} // download file
PATCH  /api/files/{id}/rename // rename file/folder
DELETE /api/files/{id}        // delete file/folder

POST   /api/shares            // grant permission
DELETE /api/shares/{shareId} // revoke permission
```

Ашигласан материалын жагсаалт

[1] Threadgold consulting. *Personal Cloud Storage Usage*. Personal Cloud Storage Usage report. 2025. URL: https://threadgoldconsulting.com/research/personal-cloud-storage-usage (**urlseen** 01/03/2026).

[2] Dean Hildebrand, Denis Serenyi. *Colossus under the hood: a peek into Google's scalable storage system*. Google's Scalable Storage Solution. 2021. URL: https://cloud.google.com/blog/products/storage-data-transfer/a-peek-behind-colossus-googles-file-system (**urlseen** 28/03/2026).

[3] Google Workspace. *Google Workspace security whitepaper: Technology with security at its core*. Google Workspace Security Whitepaper. 2026. URL: https://workspace.google.com/learn-more/security/security-whitepaper/page-4/ (**urlseen** 29/03/2026).

[4] Google Developers. *Google Drive API: Track changes for users and shared drives*. Google Drive API Documentation. 2026. URL: https://developers.google.com/workspace/drive/api/guides/manage-changes (**urlseen** 29/03/2026).

[5] Google Developers. *Google Drive API: Receive notifications for resource changes*. Google Drive API Push Notification Guide. 2026. URL: https://developers.google.com/workspace/drive/api/guides/push (**urlseen** 29/03/2026).

[6] Google Developers. *Google Drive API: Manage uploads*. Google Drive API Upload Guide. 2026. URL: https://developers.google.com/drive/api/v3/manage-uploads (**urlseen** 29/03/2026).

[7] Apple. *iCloud security overview*. Apple Platform Security. 2022. URL: https://support.apple.com/en-au/guide/security/secacde2d0da/web (**urlseen** 29/03/2026).

[8] Apple Developer. *Build apps using CloudKit*. CloudKit Overview and Architecture Concepts. 2026. URL: https://developer.apple.com/icloud/cloudkit/ (**urlseen** 29/03/2026).

[9] Apple Support. *iCloud data security overview*. Apple iCloud Data Security Categories and CloudKit Security Diagram. 2026. URL: https://support.apple.com/en-us/102651 (**urlseen** 29/03/2026).

[10] Microsoft. *Data encryption in OneDrive and SharePoint*. Microsoft Learn Documentation. 2025. URL: https://learn.microsoft.com/en-us/purview/data-encryption-in-odb-and-spo (**urlseen** 29/03/2026).

[11] Microsoft. *Overview of OneDrive in Microsoft 365*. Microsoft Learn Documentation. 2025. URL: https://learn.microsoft.com/en-us/sharepoint/onedrive-overview (**urlseen** 29/03/2026).

[12] Microsoft Support. *How OneDrive safeguards your data in the cloud*. OneDrive Security and Resilience Practices. 2026. URL: https://support.microsoft.com/en-us/office/how-onedrive-safeguards-your-data-in-the-cloud-23c6ea94-3608-48d7-8bf0-80e142edd1e1 (**urlseen** 29/03/2026).

[13] Microsoft. *How OneDrive sync works (PDF)*. OneDrive Sync Process Architecture Diagram. 2019. URL: https://download.microsoft.com/download/B/E/3/BE3AA684-77D3-4985-B0A2-87775F21CF31/How%20OneDrive%20sync%20works.pdf (**urlseen** 29/03/2026).

[14] Nextcloud GmbH. *Nextcloud Architecture Whitepaper*. Whitepaper. Accessed: 2026-03-01. Nextcloud, **july** 2018. URL: https://nextcloud.com/c/uploads/2022/03/Architecture-Whitepaper-WebVersion-072018.pdf.

[15] Nextcloud. *Nextcloud System Requirements*. NextCloud official Documentation. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/installation/system_requirements.html (**urlseen** 28/03/2026).

[16] Syncthing Project. *Syncthing Protocol Specifications*. Syncthing Documentation. 2024. URL: https://docs.syncthing.net/specs/index.html (**urlseen** 01/03/2026).

[17] IBM. *What is end-to-end encryption (E2EE)?* Information about E2EE. 2026. URL: https://www.ibm.com/think/topics/end-to-end-encryption (**urlseen** 01/03/2026).

[18] Rust. *The Rust Programming Language*. Book about Rust. 2026. URL: https://doc.rust-lang.org/book/ (**urlseen** 02/03/2026).